Disclosure Boundaries

A Threat Model and Leakage Formalisation
for Privacy Gateways in LLM Agent Systems

Eunice Lee Sophia Shim Caleb Lee

J1729Labs

Correspondence: director@j1729labs.online

September 2026

Abstract

Systems that place a cryptographic gateway between an enterprise data estate and a third-party large language model are routinely described as protecting "the data" from "the provider." That description names one adversary where there are three, and it misses what is distinctive about the model host: it observes the interface and also proposes the queries that cross it.

We separate the model host, the infrastructure operator and the legitimate tool caller, characterising each by an observation function together with an *influence* parameter γ — the worst-case probability that a call it proposes is executed. We define the disclosure boundary as a partition of system state whose every plaintext crossing is a gated, audited operation; we decompose gateway leakage into catalogue, token, disclosed-value and access-pattern components; and we prove simulation-based security statements for the host and for the operator. Those statements are independent of γ, which is the point: influence governs how quickly an adversary steers the leakage, not what leaks.

The main results are negative and concern composition. Total leakage is provably *sub*-additive for deterministic views, so bit-counting disclosure budgets are unsound. What fails is safety. We exhibit an instance in which k-anonymity and minimum-query-set-size control both hold and each view alone leaves the adversary's posterior on a target record unchanged, yet the joint view reconstructs that record exactly in two admitted aggregate queries. The token channel is not an additional summand but the *enabling condition* for the aggregate channel, because the gateway's predicate language is written over tokens the adversary must first observe. Finally, minimum-query-set-size control bounds row weights of the released query matrix while determination is a property of its row space, and the rank of that matrix is not the right budget either. The control the characterisation licenses is a span condition on the row space, whose single-record case is enforceable in polynomial time and whose general case is a minimum-distance question.

We state precisely what the model does not protect, and derive two artefacts for practice: a constraint list for gateway tool schemas, and a descriptive mapping from each mechanism to Korean and European regulatory obligations. We make no claim that the residual leakage is small; quantification is the subject of companion work.

Keywords: leakage-parameterised security; disclosure boundary; LLM agents; tool protocols; inference control; structured encryption; tokenisation

1 Introduction

An enterprise that wants to apply a commercial large language model (LLM) to its own records faces a problem that is easy to state and hard to specify. The records are subject to contractual and statutory confidentiality obligations; the model is operated by a third party; and useful work requires that something about the records reach the model. The deployment pattern that has emerged in response is a *privacy gateway*: a component sitting between the data estate and the model that substitutes tokens for sensitive values, executes aggregate computations over ciphertexts, and mediates every tool call the model makes back into the estate. Vendors describe such gateways as ensuring that "the provider never sees your data."

That sentence is not a threat model. It names one adversary where there are at least three, and it names the wrong capability for the adversary it does name.

1.1 Three adversaries, not one

Consider what actually observes what in such a deployment.

The **model host** sees the prompts sent to it, the completions it produces, the arguments of every tool call it emits, and every value returned by those calls. It can correlate across sessions and across tenants. It holds no decryption key. Crucially, it is not a passive recipient of a query stream chosen by an honest client: the tool calls *are* its outputs. It proposes what is asked next.

The **infrastructure operator** — a cloud provider, a colocation tenant, a database administrator in a managed-service arrangement — sees stored ciphertexts, the physical access sequence over them, object sizes and timings. It holds no key either, and it never sees a prompt. Its attack surface is the access-pattern and volume leakage that the searchable-encryption literature has studied for a decade [9, 38, 40, 43].

The **legitimate tool caller** — an analyst, a call-centre agent, a downstream service — holds valid permissions and sees whatever the gateway is willing to disclose to a principal with those permissions. Its attack is not a break of confidentiality but an abuse of it: repeated aggregate queries whose answers, in combination, determine an individual value. This is precisely the statistical-database inference problem posed by Denning et al. in 1979 [20].

These adversaries differ in what they observe, in what they can influence, and in which gateway module is supposed to constrain them. A design document that speaks of "the attacker" selects, implicitly and usually unconsciously, one of them, and then claims a property that is meaningless for the other two. Worse, the composite adversary — a model host colluding with, or simply manipulating, a legitimate caller — is the one most likely to occur in practice and the one least often analysed.

1.2 The host is also the query planner

The encrypted-database literature models adaptivity carefully: in the chosen-query games of Curtmola et al. [17] and Chase and Kamara [10] the adversary does select its own queries, and the simulator must answer without knowing what comes next. What those games do not model is *refusal*. The adversary's chosen query is always executed, because in the deployments they abstract — an honest client querying an outsourced index — there is nobody between the query and the data. Adversarial influence over the *data* has been studied, notably by Zhang et al. [62], who showed that an adversary able to insert documents into an encrypted index recovers queries with remarkable efficiency.

An LLM gateway inverts that picture. The adversary does not inject data; it emits the queries. A tool call is a token sequence produced by the model host. Whether that call executes depends on an approval path — policy evaluation inside the gateway, and, in many deployments, a human or

an orchestrator outside it — but the *proposal* is entirely the host's. And the approval path is itself a target: indirect prompt injection is, viewed from this angle, a technique for raising the probability that an adversary-chosen call is approved [18, 36].

We therefore model an adversary not by observation alone but by a pair: an observation function, and an *influence* parameter γ, the worst-case probability that a call it proposes survives the approval path. The textbook adaptive adversary has $\gamma = 1$; a gateway with a working approval path sits strictly below it; and the searchable-encryption server sits outside the scale entirely, since it proposes nothing. This parameter separates two questions that are usually run together: what a deployment leaks, and how fast an adversary can steer the leakage somewhere useful. The answer, Corollary 35, is that it governs only the second.

1.3 Leakage does not "add up" — something worse happens

Practitioners who take leakage seriously reach naturally for an additive accounting: so many bits from tokenisation, so many from released aggregates, sum them, compare against a budget. Our first observation is that this is formally backwards. Because both the token view and the aggregate view are deterministic functions of the database, their joint mutual information with the database is *at most* the sum of the marginals, with the excess given exactly by their mutual redundancy (Proposition 36). Total information is sub-additive; super-additive leakage, in the information-theoretic sense, cannot occur here.

What fails is not the arithmetic of bits but the composition of *safety predicates*. We construct an instance (Theorem 41) in which the token view satisfies k-anonymity with respect to the declared quasi-identifiers for arbitrarily large k, every released aggregate satisfies a minimum-query-set-size control with the same k, and from either view alone the adversary's posterior on a target record's sensitive value is exactly its prior. Both stated guarantees hold throughout, and neither channel considered on its own says anything about the value in question. From the joint view, nonetheless, the adversary reconstructs that value *exactly*, using two queries the admission control admits — four, if a pairwise set-difference control is added on top.

The mechanism is structural rather than incidental. A gateway's aggregate interface accepts predicates written over tokens. An adversary that has not observed tokens has no predicate vocabulary at all — guessing an ℓ-bit token succeeds with probability $2^{-\ell}$ — so the aggregate channel is unusable. The token channel supplies the vocabulary that makes the aggregate channel exploitable. It is the enabling condition, not an additional summand. Where the catalogue also publishes attributes in the clear, the enabling object is the exposed vocabulary as a whole (Remark 48); the pure case is the one tokenisation gateways are sold as achieving.

Composition failure is not itself new — Ganta et al. [33] showed that independently k-anonymous releases can be intersected to re-identify — and we claim novelty only for the mechanism and for its consequence that the two channels cannot be budgeted separately even in principle.

Finally we show that the most widely deployed aggregate-side control is provably incapable of bounding reconstruction. Minimum-query-set-size control constrains the *row weights* of the matrix of released query sets; reconstruction is governed by its *row space* (Theorem 49). These are independent: for every $k \leq n/2$ there is an $n \times n$ zero-one matrix whose every row has weight exactly k and whose rank is n (Theorem 51).

It would be natural to conclude that the budget unit is therefore the rank of that matrix. It is not, and saying why is the most useful thing this paper does for an implementer: the two-query attack above produces a matrix of rank two, so any rank budget worth having admits it. Rank is a dimension, whereas Theorem 49 is a statement about *which* vectors the row space contains. The

control the characterisation actually licenses is the row-space lift of the size control itself — admit only while every non-zero vector of the row space has support at least k (Definition 54) — whose single-record case is enforceable in polynomial time and whose general case sits adjacent to problems known to be NP-hard (Remark 58).

1.4 Contributions

A party and trust model. Section 3 separates the six roles a gateway deployment actually contains and fixes, for each, what it observes and what it is assumed not to do. The gateway itself is decomposed into three modules — semantic, compute, and admission — and each is mapped to the adversary it is responsible for (Table 2). The mapping is as much a warning as a design: it makes visible that no module is responsible for the composite adversary, which is the gap the rest of the paper exploits.

γ-influence adaptivity. Definition 5 separates observation from the reliability with which a chosen query is executed. The separation earns its keep immediately: our simulation theorems hold for every γ, while the number of proposals an attack needs grows by at most $1/\gamma$ (Corollary 35). Approval paths, in other words, are rate limiters rather than confidentiality mechanisms, and a deployment that treats human confirmation as the latter has misread what it bought.

The disclosure boundary as an object. Section 5 gives the boundary a definition rather than a picture: a partition of system state, an explicit gated and audited Reveal operation, a disclosure trace, and a four-component leakage decomposition. We prove simulation-based security statements for the host and infrastructure adversaries (Theorems 28 and 29). One condition inherited from simulatable auditing [44] deserves early mention, because it is routinely omitted in practice: admission *denials* are disclosure and must be accounted as such.

Composition results. Section 6 contains the paper's substance — the sub-additivity identity (Proposition 36), the safety separation (Theorem 41), the vocabulary-enabling theorem (Theorem 46), the row-space criterion and the collapse of minimum-query-set-size control (Theorems 49 and 51), and the span condition that replaces both that control and the tempting rank budget (Definition 54 and Proposition 55), together with its complexity dichotomy (Remark 58). These results are stated negatively on purpose. Each says that a control a deployment believes it has does not bound the quantity it believes it bounds.

Artefacts for practice. Three tables and one constraint list are written to be lifted directly into deployment documentation and interface specifications: the adversary definition table (Table 1), the disclosure-boundary mapping (Table 3), fourteen tool-schema constraints that a conforming gateway interface must satisfy (Appendix A), and a descriptive regulatory mapping for Korean and European obligations (Appendix B).

1.5 What this paper deliberately does not do

It does not claim that the residual leakage is small. Every result here is of the form "exactly this is exposed" or "this control does not bound that quantity." Measuring how much of the exposed leakage is exploitable against realistic corpora — in particular against the low-cardinality business-entity populations typical of Korean B2B deployments — requires an implementation and an experimental corpus, and is the subject of companion work. Readers looking for a favourable number will not find one; readers looking for a statement of what would have to be measured will find it in Section 6.6.

It also does not propose a new cryptographic construction. The gateway is assumed to be built from a scoped pseudorandom tokeniser and an additively homomorphic encryption scheme with the properties stated in Section 3.3; any scheme meeting them will do.

2 Background and Related Work

2.1 Leakage-parameterised security

Modern treatments of encrypted search do not claim that nothing leaks. They parameterise security by an explicit leakage function and prove that the adversary's view is simulatable from that function alone. Curtmola et al. [17] gave the first such definitions for searchable symmetric encryption; Chase and Kamara [10] generalised them to structured encryption with *controlled disclosure*, in which parts of a structure are revealed deliberately under a policy. Our disclosure boundary is a direct descendant of that idea, and our $\mathcal{L}$-adaptive definitions follow their template. Kamara et al. [42] extended the framework to dynamic data; Fuller et al. [32] systematised the resulting design space.

Two aspects of that literature transfer directly. First, leakage must be written down as a function, not described in prose. Second, a scheme is not "secure" or "insecure" but secure *relative to* a stated leakage profile, which leaves the question of exploitability open by design.

2.2 Leakage-abuse attacks

That open question has been answered repeatedly in the negative. Islam et al. [40] recovered queries from access patterns; Cash et al. [9] formalised leakage-abuse attacks and demonstrated them against deployed constructions; Naveed et al. [51] recovered plaintext columns from property-preserving encrypted databases using only public auxiliary distributions; Kellaris et al. [43] gave generic reconstruction from access-pattern and communication-volume leakage; Grubbs et al. attacked order-revealing encryption [37] and reconstructed databases from volume leakage on range queries [38]. Blackstone et al. [6] revisited the attack literature and showed how sharply attack efficacy depends on the auxiliary-knowledge assumptions. Bindschaedler et al. [5] made the same point from a statistical-inference perspective.

The lesson we take is methodological. An honest threat model states the leakage profile and *separately* states that the exploitability of that profile is an empirical question governed by the auxiliary knowledge available in the deployment. Conflating the two is how a design document turns into marketing.

2.3 Inference control in statistical databases

The aggregate side of an LLM gateway is a statistical database with a modern interface, and the attacks against it are not new. Denning, Denning, and Schwartz [20] introduced trackers: sequences of queries, each individually satisfying a minimum-query-set-size control, whose answers combine to yield an individual statistic. Denning and Schlörer [19] and Adam and Worthmann [1] surveyed the resulting control methods and their failures. Dinur and Nissim [21] proved that a database of n entries is reconstructable from $O(n \log^2 n)$ subset sums perturbed by $o(\sqrt{n})$ noise — the result that eventually motivated differential privacy [23, 24, 26]. Dwork and Yekhanin [25] strengthened the attacks further.

Dobkin, Jones, and Lipton [22] gave the first bounds on how many set-size-controlled queries suffice to compromise a record, and the *auditing* line that followed — Chin and Özsoyoğlu [14], and Kleinberg, Papadimitriou, and Raghavan [47] — studied whether a monitor can decide, from

the history of answered queries, that the next one is safe. The answer depends sharply on the value domain: over the rationals the decision is a rank test and is polynomial (Theorem 49), while for *Boolean* attributes it is NP-hard [47].

Three results from this literature are load-bearing for us. The tracker construction underlies Theorem 41. The linear-algebraic view of compromise [14, 21] underlies Theorem 49, which we restate in gateway terms rather than claim as new. And the auditing literature is what calibrates Remark 58. Finally, Kenthapadi, Mishra, and Nissim's *simulatable auditing* [44] establishes that refusing a query can itself disclose information, because the refusal depends on the data — a point we lift into a well-formedness condition on the gateway (Definition 17, W5) and into the definition of $\mathcal{L}_{\mathsf{agg}}$.

2.4 Anonymisation and its composition failures

k-anonymity [59] and its refinements [49] remain the operative vocabulary in regulated deployments, largely because pseudonymisation regimes are written in similar terms. Their composition behaviour is poor and well documented. Ganta, Kasiviswanathan, and Smith [33] showed that two independently k-anonymous releases of overlapping populations can be intersected to re-identify individuals — the closest antecedent of our Theorem 41, and the reason we claim novelty only for the specific mechanism by which the two channels of a gateway interact, not for composition failure as a phenomenon. Kifer and Machanavajjhala [45] made the separate and more general point that no privacy guarantee of this kind holds without assumptions on the data-generating distribution, which is why Construction 39 states its prior explicitly. And k-anonymity offers no protection against value disclosure when the sensitive attribute is homogeneous within a class [49]. We use k-anonymity here only as a *stated deployment predicate* whose non-composition with aggregate controls we then demonstrate; we are not proposing it as a guarantee.

2.5 Privacy technology for language models

Work on protecting data from model providers falls into three groups. *Sanitisation* approaches rewrite prompts before transmission, substituting or generalising entities and restoring them on return [12, 27]; they are the ancestors of the semantic module considered here. *Encrypted inference* evaluates the model itself under homomorphic encryption or secure multi-party computation [11, 39, 48]; the cost remains far from practical for frontier-scale models, and, more importantly, it addresses a different deployment in which the model weights, not the enterprise data, drive the architecture. *Confidential computing* places inference inside a hardware enclave [16], substituting a hardware trust assumption for a cryptographic one.

Orthogonally, a substantial literature measures what models expose about their own training data [8, 46, 57], and surveys [61] catalogue the space. None of this work models the tool-calling loop, and none treats the model host as a query planner.

2.6 Agentic systems and tool protocols

Tool-use protocols such as the Model Context Protocol [2] have standardised the interface through which a model invokes enterprise functions, which is exactly the interface a gateway must mediate. The security literature on these systems is dominated by indirect prompt injection [18, 36] and by practitioner taxonomies such as the OWASP list for LLM applications [52]. That work asks whether an agent can be induced to take an unintended *action*. We ask a complementary question: given that it can, what does the resulting query sequence *disclose*, and which of the gateway's controls actually

bounds that disclosure.

2.7 Positioning

We claim novelty narrowly, and state the qualifications. For *LLM agent gateways*, we are not aware of prior work that separates the model host, the infrastructure operator, and the legitimate caller as distinct adversaries with distinct leakage profiles in a single formal model; the outsourced-database literature does separate adversaries by observation [3, 32, 54], and our contribution is the transfer, not the idea. We are not aware of a prior definition in which the *probability that an adversary-chosen query is executed* appears as a parameter; note that adaptive adversaries in searchable encryption [10, 17, 42] do choose their own queries, so what is new is the gating, not adaptivity. And the composition failure we exhibit is an instance of a known phenomenon [33]; what we believe is new is the identification of the predicate vocabulary as the channel through which the interaction occurs, and the consequence that the two channels cannot be budgeted independently even in principle.

The remaining ingredients — $\mathcal{L}$-adaptive simulation, trackers, the linear-algebraic characterisation of compromise — are classical, and we use them deliberately so that the resulting model is comparable with existing security literature rather than a private formalism.

3 System Model

3.1 Parties and trust assumptions

Fig. 1 fixes the setting. We distinguish six roles.

Data owner DO**.** Holds the plaintext database D, the tokenisation key, and the decryption key. It is trusted by definition: it is the principal on whose behalf confidentiality is asserted, so a model in which it is adversarial has no statement left to make.

Gateway G**.** Runs inside DO's trust domain and holds key material, and comprises the three modules of Section 3.4. We assume it is honest and correctly implemented. That assumption is load-bearing and we flag it here rather than in a footnote: every result below is a statement about what a correctly functioning gateway discloses, and gateway compromise is out of scope (Section 7).

Model host H**.** Operates the language model, holds no key material, and observes every prompt, completion, tool-call argument and tool return. We assume it is *honest-but-curious with respect to protocol execution* but *arbitrary with respect to query selection*: it answers prompts however it likes, including in ways designed to elicit particular subsequent calls. This split is unusual and is the reason the model needs two parameters rather than one; Definition 5 makes it precise.

Infrastructure operator I**.** Hosts the ciphertext store and the network path, holds no key material, and never sees a prompt. What it does see — stored objects, the physical access sequence, object volumes and timings — is a profile with a mature attack literature behind it, and the gateway does nothing to suppress it.

End user U**.** A principal with a permission set $\mathsf{perm}(\mathsf{U})$, seeing whatever values those permissions disclose. It may be adversarial within its permissions, which is a different thing from being an intruder: its every query is legitimate, and only the sequence is not.

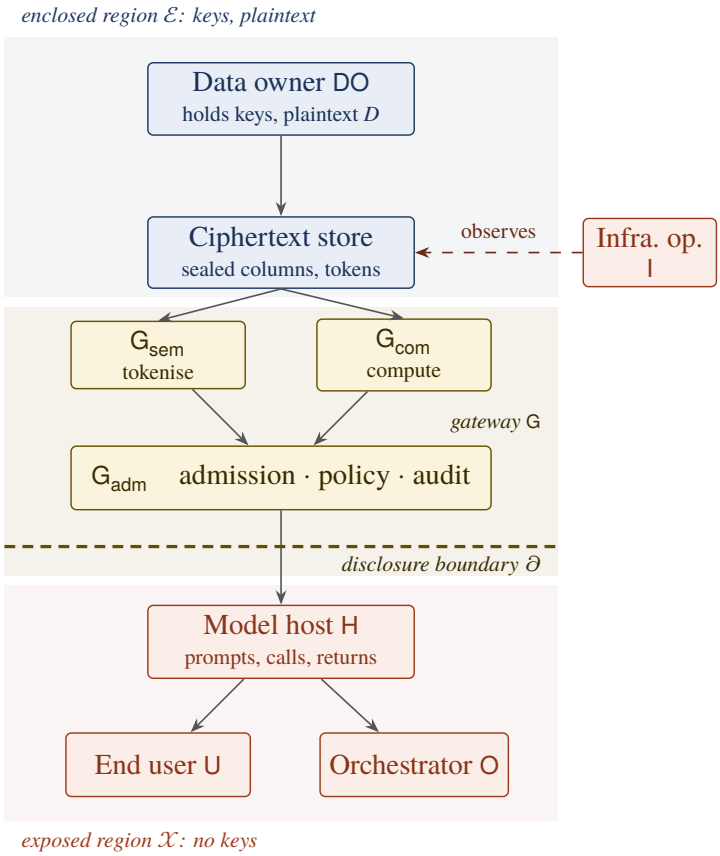


Figure 1: Parties and the disclosure boundary. The store is inside the enclosed region in the sense that it holds no plaintext, but its *operator* I observes it from outside; this asymmetry is why $\mathcal{L}_{\text{pattern}}$ must be a separate leakage component.

Orchestrator O**.** The agent loop that carries calls between H and G and implements any approval path. It may sit inside DO's trust domain when self-hosted, or outside it when the vendor runs it. We parameterise rather than assume its placement, because that placement is what determines γ — and therefore how quickly an adversary that has already been granted the interface can work through it.

3.2 Data model

Let $\Sigma = (A_1, \ldots, A_d)$ be a schema with domains $\mathrm{Dom}(A_j)$, and let $D = (r_1, \ldots, r_n)$ with $r_i \in \prod_j \mathrm{Dom}(A_j)$. We write $r_i[A_j]$ for the value of attribute A_j in record i.

Every attribute carries a *protection class* $\mathsf{cls}(A_j) \in \{\texttt{open}, \texttt{token}, \texttt{sealed}\}$, and the three classes differ in what form the attribute takes once it has left the estate. An `open` attribute is released verbatim across the boundary and is therefore public as far as the model is concerned. A `token` attribute is released as a token under the scheme of Section 3.3, which is to say it keeps its identity as a referent — equal values receive equal tokens — while losing its content. A `sealed` attribute is never released as an individual value at all: it participates only in ciphertext computation and in aggregates that cross the boundary through an explicit disclosure operation.

The classification is exhaustive by construction, and Definition 17 turns that into an obligation on the interface rather than a convention: anything appearing outside the estate must be accounted for by one of the three.

We write $\Sigma_{\texttt{open}}, \Sigma_{\texttt{token}}, \Sigma_{\texttt{sealed}}$ for the induced partition, and $\mathrm{QI} \subseteq \Sigma_{\texttt{open}} \cup \Sigma_{\texttt{token}}$ for the declared quasi-identifier set — those attributes the deployment believes also appear in an adversary's auxiliary sources. The distinction between QI and the remaining non-sealed attributes is doing real work in Section 6 and is a faithful rendering of how pseudonymisation risk assessments are actually written.

3.3 Cryptographic components

We abstract two primitives. The results do not depend on their internals.

Definition 1 (Scoped tokeniser)**.** A *scoped tokeniser* for a value domain $\mathcal{V}$ is a pair $T = (\mathsf{Gen}, \mathsf{Tok})$ where $\mathsf{Gen}(1^\kappa) \to K$ and $\mathsf{Tok}_K : \mathcal{S} \times \mathcal{V} \to \{0,1\}^\ell$ is deterministic, with $|\mathcal{V}| \le 2^\ell$ and $\ell = \omega(\log \kappa)$. The scope set is $\mathcal{S} = \Sigma_{\texttt{token}} \times \mathcal{S}_0$: a scope always names the attribute as well as the session, tenant or epoch, so distinct attributes never share a token space. T is *scope-confined* if for every PPT adversary $\mathcal{A}$ given oracle access to $\mathsf{Tok}_K(s,\cdot)$ for *every* $s \in \mathcal{S}$ simultaneously,

$$\left| \Pr\left[\mathcal{A}^{\{\mathsf{Tok}_K(s,\cdot)\}_{s\in\mathcal{S}}}(1^\kappa) = 1\right] - \Pr\left[\mathcal{A}^{\{\pi_s(\cdot)\}_{s\in\mathcal{S}}}(1^\kappa) = 1\right] \right| \le \mathsf{negl}(\kappa),$$

where the π_s are independent uniformly random injections $\mathcal{V} \to \{0,1\}^\ell$.

Remark 2 (Token length is a security parameter). The condition $\ell = \omega(\log \kappa)$ is not decorative: three statements below (Theorem 28, Theorem 46, Theorem 41(d)) bound an adversary's success by $2^{-\ell}$ terms. Format-preserving tokenisation, which replaces a value by a string of the same shape so that legacy systems continue to parse it, typically yields ℓ of thirty to forty bits and *violates* the condition. Such a deployment does not satisfy the hypotheses of our theorems, and a gateway using it should say so.

A scope-confined tokeniser hides everything about its inputs except the *equality pattern within a scope*: this is the same guarantee that deterministic encryption provides in the sense of Bellare et al. [4], and it is the whole of what $\mathcal{L}_{\mathsf{token}}$ will capture. The scope function is a deployment choice — $\mathcal{S}$ may be a singleton (global tokens, maximal linkability, maximal utility across sessions), one scope per session, or one per epoch — and Section 6 shows that this choice, not the strength of the tokeniser, is the parameter that matters.

Definition 3 (Additively homomorphic scheme)**.** $E = (\mathsf{KGen}, \mathsf{Enc}, \mathsf{Dec}, \boxplus)$ is IND-CPA secure and satisfies $\mathsf{Dec}_{sk}(\mathsf{Enc}_{pk}(x) \boxplus \mathsf{Enc}_{pk}(y)) = x + y$ over the plaintext group.

The gateway of interest is built on elliptic-curve ElGamal in exponential form [28], which meets Definition 3 under DDH; schemes supporting a bounded multiplicative depth [7] or arbitrary depth [13, 34] also qualify, and nothing below depends on which. What *does* matter is that decryption is exact over the integers: Theorem 49 is a statement about an exactly determined linear system, and approximate schemes [13] change the analysis in the direction studied by Dinur and Nissim [21].

3.4 Gateway architecture

G consists of three modules with disjoint responsibilities.

Semantic module $\mathsf{G}_{\mathsf{sem}}$**.** Applies Tok on egress and Tok^{-1} on ingress for attributes in $\Sigma_{\texttt{token}}$, and performs egress screening of free-text fields. Its adversary is $\mathcal{A}_{\mathsf{host}}$: it exists so that the model host receives referents rather than values.

Compute module $\mathsf{G}_{\mathsf{com}}$**.** Evaluates aggregate functions over $\Sigma_{\texttt{sealed}}$ attributes homomorphically, so that the store never holds and the operator never observes a plaintext sensitive value. Its adversary is $\mathcal{A}_{\mathsf{infra}}$.

Admission module $\mathsf{G}_{\mathsf{adm}}$**.** Decides, for each proposed call, whether it executes; maintains per-principal budget state; and emits audit records. Its adversary is $\mathcal{A}_{\mathsf{user}}$ — and, as Section 6 shows, it is also the only module positioned to address the composite adversary, which is why its budget unit matters so much.

3.5 Interface and execution model

G publishes a tool set $\mathcal{T}$. A *call* is a pair $c = (\mathsf{name}, \mathsf{args})$ with $\mathsf{name} \in \mathcal{T}$. We distinguish three tool families, which differ in what they place outside the estate.

Lookup tools. These return projections of records onto $\Sigma_{\texttt{open}} \cup \Sigma_{\texttt{token}}$, with `token`-class values tokenised. They are the cheap operations of the interface, and are usually treated as such in deployment: they disclose no sealed value, so they are rarely budgeted. Section 6 is largely an argument that this accounting is wrong.

Aggregate tools. These take a predicate P drawn from a declared predicate language $\mathcal{P}$ together with a function symbol f, and return $f(\{r_i[A] : P(r_i)\})$ for a `sealed` attribute A, computed under $\boxplus$ and decrypted at the boundary. They are what makes the architecture useful — the model can ask about the data without receiving it — and also what makes it a statistical database, with the inference problems that implies.

Reveal tools. These disclose a named individual value across the boundary, under the gate and audit obligations of Definition 15. They exist so that the deliberate disclosures a business genuinely needs are visible as such rather than arriving as side effects of something else.

Crucially, $\mathcal{P}$ is written over the *exposed* vocabulary: a predicate may compare a `token`-class attribute to a token, or an `open`-class attribute to a literal, but it cannot name a plaintext value of a `token`- or `sealed`-class attribute. This is not a modelling convenience — it is forced, since the model host has never seen such a value. It becomes the hinge of Theorem 46.

The execution model is a loop. At step j, given its view so far, H emits a proposed call c_j. An approval predicate Φ, implemented by O and possibly involving U, returns allow or deny. If allowed, $\mathsf{G}_{\mathsf{adm}}$ evaluates policy Π and budget state and returns admit or reject. If admitted, G executes c_j and returns resp_j; otherwise it returns a rejection notice. Both outcomes are appended to H's view, and — as Section 5.4 makes precise — both are leakage.

4 Adversaries

An adversary in this setting is characterised by two independent attributes: what it *observes*, and how reliably a query it chooses is actually executed. The encrypted-database literature varies the first richly and treats the second as all-or-nothing — either the client chooses the queries or the adversary does; the agent-security literature studies the second and largely ignores the first. Treating them as independent coordinates is the organising move of this section.

4.1 Observation

Definition 4 (View)**.** Let $Q = (c_1, \ldots, c_q)$ be the sequence of executed calls in an execution of the gateway on database D. The *view* of a party is the transcript of everything it receives during that

execution, together with its own inputs and randomness. We write $\mathsf{View}_{\mathsf{H}}$, $\mathsf{View}_{\mathsf{I}}$, $\mathsf{View}_{\mathsf{U}}$.

Concretely:

$$\mathsf{View}_{\mathsf{H}} = \big(\Sigma^{\mathrm{pub}},\ \{(c_j, \mathsf{dec}_j, \mathsf{resp}_j, t_j)\}_{j \le q},\ \mathsf{prompts},\ \mathsf{completions}\big),$$

where Σ^{pub} is the published catalogue, dec_j the admit/reject decision, and t_j an API-granularity timestamp;

$$\mathsf{View}_{\mathsf{I}} = \big(\Sigma^{\mathrm{pub}},\ \mathcal{C},\ \mathsf{acc},\ \mathsf{vol},\ \mathsf{tim}\big),$$

where $\mathcal{C}$ is the stored ciphertext collection, acc the physical access sequence, vol object and response volumes, and tim storage-layer timings; and $\mathsf{View}_{\mathsf{U}}$ contains the rendered outputs the user's permissions entitle it to, including disclosed values.

4.2 Influence

Definition 5 (γ-influence)**.** The *influence* of an adversary $\mathcal{A}$ with respect to an approval path $(\Phi, \mathsf{G}_{\mathsf{adm}})$ is

$$\gamma(\mathcal{A}) \;=\; \inf_{c,\,v} \Pr\big[\Phi(c) = \mathsf{allow} \;\wedge\; \mathsf{G}_{\mathsf{adm}}(c) = \mathsf{admit} \;\big|\; \mathsf{View}_{\mathcal{A}} = v\big],$$

the infimum over calls c that $\mathcal{A}$ proposes with positive probability and views v at which it proposes them, the probability taken over the randomness of the approval path and the environment. We call $\mathcal{A}$ *fully adaptive* if $\gamma(\mathcal{A}) = 1$, and *passive* if $\mathcal{A}$ proposes no calls at all.

Remark 6. Passivity is a property of the adversary — it proposes nothing — and not a value of γ, which for such an adversary is an infimum over an empty set. We write $\gamma = 0$ for the passive case as a notational convenience only.

Remark 7. The passive case is the server of searchable encryption: it observes a query stream chosen entirely by an honest client and proposes nothing. $\gamma = 1$ is the textbook adaptive adversary of $\mathcal{L}$-adaptive definitions [10, 17], which selects queries freely and has every one executed. A deployed gateway lies strictly between: the adversary selects, but the approval path may refuse. It is the refusal probability, not the selection, that is new here. γ is a property of the *deployment*, not of the cryptography: an auto-approving orchestrator with a permissive policy gives γ near 1; a human confirming each call on a narrow tool surface gives γ close to 0. Indirect prompt injection [18, 36] is, in these terms, a family of techniques for raising γ by manipulating Φ rather than by breaking any cryptographic assumption.

Remark 8 (Relation to injection attacks). Zhang et al. [62] give an adversary that influences the *data* in an encrypted index and thereby recovers queries. Our $\mathcal{A}_{\mathsf{host}}$ influences the *queries* and thereby recovers data. The two are duals. Query *selection* by the adversary is of course standard in adaptive definitions; what we have not seen treated as a definitional parameter is the *probability* that a selected query survives an approval path.

4.3 The three adversaries

Definition 9 ($\mathcal{A}_{\mathsf{host}}$)**.** $\mathcal{A}_{\mathsf{host}}$ is the adversary with $\mathsf{View}_{\mathsf{H}}$ and influence γ_{H} determined by the deployment's approval path. It holds no key material and does not deviate from the protocol as a *responder*; it is unconstrained as a *proposer*. It may correlate across sessions and, where the same host serves several tenants, across tenants.

Table 1: Adversary definitions. "Influence" is γ of Definition 5. No adversary holds key material.

Adversary	Observes	Does not observe	Influence	Primary attack	Constraining module
$\mathcal{A}_{\text{host}}$ *model host*	prompts, completions, tool-call arguments and returns, admit/reject decisions, API timing; cross-session and, if scope is global, cross-tenant linkage	storage layout, physical access pattern, key material, plaintext of `token`/`sealed` attributes	γ_{H}: deployment-determined; $\to 1$ under successful prompt injection	token-graph linkage with auxiliary knowledge; vocabulary-enabled aggregate reconstruction	G_{sem} (scope, egress screening)
$\mathcal{A}_{\text{infra}}$ *infrastructure operator*	stored ciphertexts, physical access sequence, object and response volumes, storage timing	prompts, completions, tool semantics, key material	passive	access-pattern, volume, and stored-token frequency inference [38, 40, 43, 51]	G_{com} (ciphertext-only storage)
$\mathcal{A}_{\text{user}}$ *legitimate caller*	all values its permissions disclose; admit/reject decisions on its own calls	anything outside its permission set; key material	γ_{U}: permission- and policy-determined	tracker and differencing attacks on aggregates [20, 21]	G_{adm} (admission, budget)
$\mathcal{A}^{+}_{\text{host}}$ *composite*	union of $\mathcal{A}_{\text{host}}$ and $\mathcal{A}_{\text{user}}$	key material, storage layer	$\geq \max(\gamma_{\text{H}}, \gamma_{\text{U}})$	host supplies token vocabulary, user's permissions admit the aggregates (Theorem 41)	none in isolation; requires joint accounting

The last clause deserves emphasis because it is often assumed away. Scope-confined tokenisation limits linkage to within a scope; if the scope is global, cross-session and cross-tenant correlation is exactly what the design permits, and no amount of key strength changes this.

Definition 10 ($\mathcal{A}_{\text{infra}}$)**.** $\mathcal{A}_{\text{infra}}$ is the passive adversary with $\mathsf{View}_{\mathsf{I}}$: it observes storage and proposes no calls. It holds no key material.

Definition 11 ($\mathcal{A}_{\text{user}}$)**.** $\mathcal{A}_{\text{user}}$ is a principal holding a valid permission set, with the view $\mathsf{View}_{\mathsf{U}}$ and influence γ_{U} given by Definition 5 over the calls its permissions allow. Its objective is value disclosure: to determine $r_{i^*}[A]$ for a `sealed` attribute A and some target i^*, using only admitted queries.

Definition 12 (Composite adversary)**.** $\mathcal{A}^{+}_{\text{host}} = \mathcal{A}_{\text{host}} \cup \mathcal{A}_{\text{user}}$ is the adversary with the union of the two views and influence at least $\max(\gamma_{\text{H}}, \gamma_{\text{U}})$.

$\mathcal{A}^{+}_{\text{host}}$ is the realistic worst case, and it is realistic for a mundane reason: the model host *writes the calls that the legitimate user approves*. An analyst who accepts an agent's suggested follow-up query is lending the host its permissions. The composite is not a collusion assumption; it is the default behaviour of the deployment.

Observation 13 (Adversary order). Order adversaries by view inclusion. Then $\mathcal{A}_{\text{host}}$ and $\mathcal{A}_{\text{infra}}$ are incomparable — neither view contains the other — which is precisely why a single "the attacker" is not well defined for this architecture. $\mathcal{A}_{\text{user}} \leq \mathcal{A}^{+}_{\text{host}}$ and $\mathcal{A}_{\text{host}} \leq \mathcal{A}^{+}_{\text{host}}$.

Table 1 collects these definitions in the form we use operationally. Its last row is the paper's practical warning: each of the three modules is sound against the adversary it was designed for, and no module is responsible for the composite.

4.4 Module-to-adversary mapping

Table 2 is the table we recommend reproducing verbatim in deployment documentation, because its third column is what a reader otherwise supplies incorrectly from context. Tokenisation is regularly

Table 2: Which module constrains which adversary, and what it does *not* do.

Module	Adversary	Mechanism	Explicit non-guarantee
$\mathsf{G}_{\mathsf{sem}}$	$\mathcal{A}_{\mathsf{host}}$	scoped tokenisation of $\Sigma_{\texttt{token}}$; egress screening of free text	does not hide equality pattern within a scope; no protection against auxiliary-knowledge linkage
$\mathsf{G}_{\mathsf{com}}$	$\mathcal{A}_{\mathsf{infra}}$	homomorphic evaluation so the store holds only ciphertext	does not hide access pattern, volume, timing, or the equality pattern of *stored* tokens
$\mathsf{G}_{\mathsf{adm}}$	$\mathcal{A}_{\mathsf{user}}$	policy gate, permission check, budget state, audit	counting queries does not bound reconstruction (Theorem 51)

read as "the host learns nothing"; it in fact provides exactly equality-pattern confinement within a scope. Homomorphic aggregation is regularly read as "the operator learns nothing"; it in fact leaves $\mathcal{L}_{\mathsf{pattern}}$ — which includes the equality pattern of every *stored* token, not only of those a query returns — entirely intact, and the attacks against that profile [9, 38, 40, 43, 51] are mature.

5 The Disclosure Boundary

5.1 State partition

Definition 14 (Enclosed and exposed regions)**.** The system state is partitioned into the *enclosed region* $\mathcal{E}$ — components that hold key material or plaintext values of $\Sigma_{\texttt{token}} \cup \Sigma_{\texttt{sealed}}$ attributes — and the *exposed region* $\mathcal{X}$, everything else. The *disclosure boundary* ∂ is the interface between them. In Fig. 1, $\mathcal{E} = \{\mathsf{DO}, \mathsf{G}\}$ and $\mathcal{X} = \{\mathsf{H}, \mathsf{O}, \mathsf{U}\}$; the ciphertext store sits in $\mathcal{E}$ by content and in $\mathcal{X}$ by observation, which is the asymmetry $\mathcal{L}_{\mathsf{pattern}}$ exists to record.

Writing the boundary as a partition rather than a picture forces a question that architecture diagrams allow one to skip: for every datum that appears in $\mathcal{X}$, by what operation did it get there? There are exactly four admissible answers — it is `open`; it is a token; it is a ciphertext; or it crossed by an explicit Reveal.

5.2 Crossing the boundary

Definition 15 (Reveal)**.** Reveal is an operation

$$\mathsf{Reveal} : (\rho,\ \mathsf{obj},\ \mathsf{purp},\ \pi) \longmapsto (v,\ a)$$

taking a principal ρ, an object reference obj (a token, a ciphertext handle, or an aggregate descriptor), a declared purpose purp, and a policy version π; returning the plaintext value v and an audit record a. Reveal succeeds only if $\Pi_\pi(\rho, \mathsf{obj}, \mathsf{purp}) = \mathsf{permit}$, and it always emits a, which records $(\rho, \mathsf{obj}, \mathsf{purp}, \pi, \mathsf{decision}, \mathsf{time})$.

Definition 16 (Disclosure trace)**.** The *disclosure trace* $\Delta(D, Q)$ of an execution is the sequence of all boundary crossings in the clear: outputs of Reveal, and the plaintext results of admitted aggregate calls, each paired with its audit record.

Two design consequences follow immediately, and both are constraints on the tool schema rather than on the cryptography. First, no tool may disclose as a side effect: if an operation can place a

plaintext sensitive value in $\mathcal{X}$, it *is* a Reveal and must be typed as one. Second, the audit record is not a logging feature but part of the security definition — Δ is the object against which a deployment can later answer "what did this system disclose?", and a crossing absent from Δ makes the question unanswerable.

5.3 Well-formed gateways

Definition 17 (Well-formed gateway)**.** A gateway G over schema Σ with tool set $\mathcal{T}$ is *well-formed* if:

(W1) *Closure.* Every value appearing in $\mathcal{X}$ is `open`-class, a token, a ciphertext, or an output of Reveal.

(W2) *Gating.* Every Reveal is preceded by a policy decision and emits an audit record.

(W3) *Typed interface.* Every tool argument and return is typed by protection class, and no free-form field carries an enclosed value without passing egress screening.

(W4) *Declared scope.* Every $A \in \Sigma_{\texttt{token}}$ declares a scope function and rotation policy.

(W5) *Simulatable admission.* The decision $\mathsf{G}_{\mathsf{adm}}(c_j)$ is computable from the exposed state $(\Sigma^{\mathrm{pub}}, \text{prior decisions}, \text{prior responses}, \Pi)$ alone.

(W6) *Uniform latency.* Response latency and error taxonomy are quantised to a declared granularity that does not depend on the data, so that the interface exposes no timing channel beyond the declared profile.

W5 is the condition most often violated in practice. A minimum-query-set-size control that rejects because the *true* query set is small has made its decision a function of the data; the rejection therefore discloses. This is the phenomenon Kenthapadi et al. [44] identified as the reason naive auditing fails. We take the conservative branch of their resolution: rather than require simulatable admission, we place the decision bits inside the leakage profile unconditionally (Definition 22), so that Theorem 28 holds whether or not W5 is satisfied. W5 then has the status of a design recommendation — a gateway that meets it discloses strictly less, because its decisions are derivable by anyone holding the rest of the profile — rather than a hypothesis of the theorem. The construction of Construction 39, whose MQSS control is data-dependent, is precisely a gateway that fails W5.

W6 is a hypothesis of Theorem 28 and is not optional: aggregate latency grows with query-set size and with the number of ciphertext additions, so an unpadded interface leaks a noisy count that no component of $\mathcal{L}_{\mathsf{host}}$ accounts for.

Remark 18. Deployments sometimes report admission denials as "no data disclosed". On the contrary: a data-dependent denial is a one-bit answer to a predicate of the analyst's choosing, and a sequence of them is a membership oracle. Definition 22 counts them, and an audit trail that records only successful calls is therefore incomplete as a disclosure record.

5.4 Leakage decomposition

Definition 19 (Catalogue leakage)**.** $\mathcal{L}_{\mathsf{schema}}(D) = \big(\Sigma^{\mathrm{pub}}, \mathsf{cls}(\cdot), \mathrm{QI}, \{\mathrm{Dom}(A_j)\}_{A_j \in \Sigma_{\mathrm{open}}}, \beta(n)\big)$, where Σ^{pub} lists attribute names and types, and $\beta(n)$ is the published cardinality bucket of D. $\mathcal{L}_{\mathsf{schema}}$ is independent of Q.

Definition 20 (Token leakage)**.** Let $\mathcal{J} = \{(j, \alpha)\}$ be the set of token-bearing slots in the responses to Q, where slot (j, α) carries token $\tau_{j,\alpha}$ for attribute A_α under scope $s_{j,\alpha}$. Define the *scoped equality*

pattern as the equivalence relation on $\mathcal{I}$

$$(j,\alpha)\ \mathsf{eq}\ (j',\alpha') \iff \tau_{j,\alpha} = \tau_{j',\alpha'}.$$

Then $\mathcal{L}_{\mathsf{token}}(D,Q)$ is the labelled hypergraph $\mathbf{H} = (V,E,\lambda)$ with $V = \mathcal{I}/\mathsf{eq}$, E the responses (each the set of classes its slots fall into), and λ labelling each incidence with its attribute and scope. $\mathcal{L}_{\mathsf{token}}$ is a function of the $\Sigma_{\texttt{open}}$ and $\Sigma_{\texttt{token}}$ values of D alone; whether it thereby carries information about $\Sigma_{\texttt{sealed}}$ depends on the correlation between those attributes in the underlying distribution, and in general it does.

Remark 21. By Definition 1 a scope always includes the attribute name, so tokens of different attributes are produced by independent injections and collide with probability at most $|\mathcal{I}|^2 2^{-\ell}$; the same holds across scopes. Hence eq coincides with the within-attribute, within-scope equality pattern except with negligible probability, and we treat them as identical throughout. A gateway that shares one token space across attributes — so that the same customer name and the same supplier name receive the same token — does not satisfy Definition 1, and its cross-attribute equalities are additional leakage.

Definition 22 (Disclosed-value and admission leakage)**.** $\mathcal{L}_{\mathsf{agg}}(D,C)$ is the pair $\big(\mathsf{dec}(C),\ \Delta(D,C)\big)$, where $C = (c_1,\dots,c_m)$ is the sequence of *proposed* calls, $\mathsf{dec}(C) = (\mathsf{dec}_1,\dots,\mathsf{dec}_m)$ the admit/reject decisions on them, and Δ the disclosure trace of Definition 16 over the admitted subsequence $Q \subseteq C$, each aggregate entry carrying its query-set descriptor P_j, function symbol f_j, and released value.

Remark 23 (Leakage is indexed by proposals, not by executions). We write every leakage component as a function of (D,C) rather than (D,Q), because Q is itself determined by dec and is therefore not available as an index before the decisions are made. Nothing changes for $\mathcal{L}_{\mathsf{token}}$ and $\mathcal{L}_{\mathsf{pattern}}$, which depend only on the admitted subsequence; the distinction matters for $\mathcal{L}_{\mathsf{agg}}$, whose first component ranges over rejections as well.

Definition 24 (Access-pattern leakage)**.** $\mathcal{L}_{\mathsf{pattern}}(D,Q) = \big(\mathcal{L}^{\mathsf{store}}_{\mathsf{token}}(D),\ \mathsf{acc},\ \mathsf{vol},\ \mathsf{tim}\big)$, where acc, vol, tim are as in Definition 4 and $\mathcal{L}^{\mathsf{store}}_{\mathsf{token}}(D)$ is the equality pattern of every token *stored* in the ciphertext store, indexed by column and scope.

Remark 25 (The stored pattern is strictly larger). $\mathcal{L}^{\mathsf{store}}_{\mathsf{token}}$ is not a special case of $\mathcal{L}_{\mathsf{token}}$: the latter covers only tokens a query has returned, while the operator sees every row of every token column whether queried or not, and hence the full frequency distribution of each column. This is precisely the input to the frequency-analysis attacks of Naveed et al. [51], and it is the component that deployments describing homomorphic storage as "the operator sees nothing" omit.

We write $\mathcal{L}_{\mathsf{host}} = (\mathcal{L}_{\mathsf{schema}}, \mathcal{L}_{\mathsf{token}}, \mathcal{L}_{\mathsf{agg}})$ and $\mathcal{L}_{\mathsf{infra}} = (\mathcal{L}_{\mathsf{schema}}, \mathcal{L}_{\mathsf{pattern}})$.

Table 3 is the second artefact intended for direct operational reuse. Its value is in the last two rows: an admission decision and a physical access are disclosures, and a system whose documentation enumerates only the first five rows has an incomplete profile.

5.5 Security definitions and statements

Definition 26 ($\mathcal{L}$-adaptive simulation security under an approval path)**.** Let G be a gateway, $\mathcal{L}$ a leakage function, $\mathcal{A}$ a PPT adversary, and Φ an approval path. Define two experiments, each parameterised by D and 1^κ, each ending by outputting $\mathcal{A}$'s entire view, including $\mathcal{A}$'s own randomness and outputs.

Table 3: Disclosure-boundary mapping. "Crosses as" names the form in which the datum appears in $\mathcal{X}$.

Class	Crosses as	Leakage component	Visible to
`open`	literal	$\mathcal{L}_{\text{schema}}$ (values are public by construction)	$\mathcal{A}_{\text{host}}$, $\mathcal{A}_{\text{user}}$
`token`	token	$\mathcal{L}_{\text{token}}$: equality pattern within scope, incidence structure	$\mathcal{A}_{\text{host}}$, $\mathcal{A}_{\text{user}}$
`sealed`	ciphertext	none at the interface; $\mathcal{L}_{\text{pattern}}$ at the store	$\mathcal{A}_{\text{infra}}$
`sealed`	aggregate value	$\mathcal{L}_{\text{agg}}$: value + query-set descriptor + decision	$\mathcal{A}_{\text{host}}$, $\mathcal{A}_{\text{user}}$
any	Reveal output	$\mathcal{L}_{\text{agg}}$ via Δ; audit record mandatory	$\mathcal{A}_{\text{host}}$, $\mathcal{A}_{\text{user}}$
—	admission decision	$\mathcal{L}_{\text{agg}}$ (see W5)	proposer
—	physical access	$\mathcal{L}_{\text{pattern}}$	$\mathcal{A}_{\text{infra}}$

$\mathbf{Real}_{\mathsf{G},\mathcal{A},\Phi}$: the data owner initialises G on D and publishes Σ^{pub}. For $j = 1, 2, \ldots$: $\mathcal{A}$ proposes c_j; Φ and G_{adm} are run on c_j, yielding dec_j; if $\mathsf{dec}_j = \mathsf{admit}$, G executes c_j and returns resp_j, otherwise it returns the rejection notice.

$\mathbf{Ideal}_{\mathcal{A},\mathcal{S},\Phi}$: $\mathcal{S}$, which may depend on Φ, receives $\mathcal{L}(D, \emptyset)$ at setup and publishes a simulated catalogue. For $j = 1, 2, \ldots$: $\mathcal{A}$ proposes c_j to $\mathcal{S}$; $\mathcal{S}$ receives $\mathcal{L}(D, C_{\leq j})$, where $C_{\leq j} = (c_1, \ldots, c_j)$ is the proposal sequence of *this* interaction, and returns the simulated pair $(\mathsf{dec}_j, \mathsf{resp}_j)$. Note that $\mathcal{L}(D, C_{\leq j})$ already contains dec_j, as the j-th entry of $\mathsf{dec}(C)$, and the response to c_j if it was admitted; this is why Definition 22 is indexed by proposals (Remark 23).

G is *$\mathcal{L}$-adaptively secure* if for every PPT $\mathcal{A}$ and every Φ there is a PPT $\mathcal{S}$ such that for all D,

$$\mathbf{Real}_{\mathsf{G},\mathcal{A},\Phi}(1^\kappa, D) \approx_c \mathbf{Ideal}_{\mathcal{A},\mathcal{S},\Phi}(1^\kappa, D).$$

Remark 27. Three points of hygiene. First, the ideal experiment is *interactive*: $\mathcal{S}$ answers $\mathcal{A}$'s adaptively chosen proposals, as in the chosen-query games of Curtmola et al. [17] and Chase and Kamara [10]. Q is meaningless otherwise, since $\mathcal{A}$ is what produces it. Second, $\mathcal{A}$'s prompts and completions are $\mathcal{A}$'s own outputs, so they are reproduced by running $\mathcal{A}$ rather than simulated — which is why both experiments output $\mathcal{A}$'s view and not a free-standing transcript. Third, Φ is a fixed randomised algorithm and γ is a quantity derived from it (Definition 5); γ appears nowhere in the definition, which is the content of Observation 34.

Theorem 28 (Host-side simulation)**.** *Let* G *satisfy W1–W4 and W6 of Definition 17, let T be scope-confined with $\ell = \omega(\log \kappa)$ (Definition 1), and let E be IND-CPA (Definition 3). Then* G *is $\mathcal{L}_{\text{host}}$-adaptively secure in the sense of Definition 26 against $\mathcal{A}_{\text{host}}$, for every approval path Φ.*

Proof. Fix $\mathcal{A} = \mathcal{A}_{\text{host}}$ and Φ, and let $q = \text{poly}(\kappa)$ bound the number of proposals. We pass through hybrids, each an interactive experiment with $\mathcal{A}$; only the way responses are produced changes.

$\mathbf{H}_0$ is $\mathbf{Real}_{\mathsf{G},\mathcal{A},\Phi}$.

$\mathbf{H}_1$ takes the admission decisions and every plaintext value crossing ∂ — aggregate results and Reveal outputs — from the corresponding entries of $\mathsf{dec}(Q)$ and $\Delta(D, Q)$, that is, of $\mathcal{L}_{\text{agg}}$; catalogue responses are produced from $\mathcal{L}_{\text{schema}}$; and by W6 the latency and error class of each response are fixed by the declared granularity. This is a change of bookkeeping, not of distribution, so $\mathbf{H}_1 \equiv \mathbf{H}_0$.

$\mathbf{H}_2$ replaces every ciphertext crossing ∂ with $\mathsf{Enc}_{pk}(0)$. The order matters: because $\mathbf{H}_1$ already reads released plaintexts from Δ rather than by decrypting, and because by W1 a ciphertext in $\mathcal{X}$ is an opaque handle, no response depends on the replaced ciphertexts' contents. A distinguisher between $\mathbf{H}_1$ and $\mathbf{H}_2$ therefore yields, by a standard hybrid over the at most $\text{poly}(\kappa)$ ciphertexts, an

IND-CPA distinguisher for E; the reduction embeds its challenge as one ciphertext and answers every aggregate query from Δ, so it never needs sk.

$\mathbf{H}_3$ replaces every token by a lazily sampled string. The simulator keeps one table per (attribute, scope) pair. To produce the token in slot (j, α) it consults $\mathcal{L}_{\mathsf{token}}(D, Q_{\leq j})$: if eq places (j, α) in a class already assigned a string, it reuses that string; otherwise it samples a fresh uniform $\{0,1\}^{\ell}$ string, resampling on collision. The result is distributed as a random injection consistent with the equality pattern, so a distinguisher between $\mathbf{H}_2$ and $\mathbf{H}_3$ contradicts scope-confinement of T up to a collision term of at most $q^2 2^{-\ell}$, negligible because $\ell = \omega(\log \kappa)$.

Every response in $\mathbf{H}_3$ is computed from $\mathcal{L}_{\mathsf{host}}$ alone, so $\mathbf{H}_3$ is $\mathbf{Ideal}_{\mathcal{A},\mathcal{S},\Phi}$ for the $\mathcal{S}$ that carries out exactly these steps, and $\mathcal{S}$ runs in polynomial time. □

Theorem 29 (Infrastructure-side simulation)**.** *Let* G *satisfy W1 and W4, let E be IND-CPA and T be scope-confined. If the store holds only* `open`*-class values, tokens, and ciphertexts, and key material never enters* $\mathcal{X}$*, then* G *is* $\mathcal{L}_{\mathsf{infra}}$*-adaptively secure against* $\mathcal{A}_{\mathsf{infra}}$*.*

Proof. $\mathcal{S}$ builds a store of the shape given by $\mathcal{L}_{\mathsf{schema}}$, filling `sealed` columns with $\mathsf{Enc}_{pk}(0)$ and each `token` column with lazily sampled strings consistent with $\mathcal{L}^{\mathsf{store}}_{\mathsf{token}}(D)$, which Definition 24 supplies, and then replays acc, vol and tim verbatim. Indistinguishability follows from IND-CPA and scope-confinement exactly as in Theorem 28. Since $\mathcal{A}_{\mathsf{infra}}$ is passive there are no proposals to answer. □

Remark 30. This proof is where Remark 25 earns its place. Without $\mathcal{L}^{\mathsf{store}}_{\mathsf{token}}$ in the profile, $\mathcal{S}$ would have to invent the stored tokens, and an adversary comparing the frequency histogram of a simulated column against a public auxiliary distribution would distinguish immediately [51]. A gateway wishing to remove this component from its profile must randomise the stored representation, which in turn breaks the equality-based joins that motivate stored tokens in the first place.

Definition 31 (Joint equality pattern)**.** $\mathcal{L}^{\sqcup}_{\mathsf{token}}(D, C)$ is the equality pattern of Definition 20 taken over the disjoint union of the response slots $\mathcal{I}$ and the token cells of the store, together with the incidence labels of both.

Corollary 32 (Composite)**.** *Under the hypotheses of Theorems 28 and 29,* G *is* $\mathcal{L}^{+}$*-adaptively secure against* $\mathcal{A}_{\mathsf{host}} \cup \mathcal{A}_{\mathsf{infra}}$*, where*

$$\mathcal{L}^{+} \;=\; (\mathcal{L}_{\mathsf{schema}},\ \mathcal{L}^{\sqcup}_{\mathsf{token}},\ \mathcal{L}_{\mathsf{agg}},\ \mathsf{acc},\ \mathsf{vol},\ \mathsf{tim}).$$

Proof. The two simulators cannot be run side by side: $\mathcal{A}_{\mathsf{host}} \cup \mathcal{A}_{\mathsf{infra}}$ sees both a returned token and the token stored in the corresponding cell, and independent lazy sampling would make them differ. We build one simulator holding a single lazy table indexed by the classes of $\mathcal{L}^{\sqcup}_{\mathsf{token}}$ and one simulated store of $\mathsf{Enc}_{pk}(0)$ ciphertexts, and serve both interfaces from it. Because $\mathcal{L}^{\sqcup}_{\mathsf{token}}$ is a single equivalence relation on $\mathcal{I} \sqcup \{\text{store cells}\}$, a response slot and a store cell receive the same string exactly when the real system would give them the same token. The hybrid argument of Theorem 28 then applies verbatim over this shared state. □

Remark 33. $\mathcal{L}^{\sqcup}_{\mathsf{token}}$ is strictly larger than the pair $(\mathcal{L}_{\mathsf{token}}, \mathcal{L}^{\mathsf{store}}_{\mathsf{token}})$: the pair records the equality pattern *within* each side, the joint object also records which response slot corresponds to which stored cell. Colluding parties learn that correspondence, and a profile that lists the two components separately understates what their union discloses — which is, in miniature, the composition problem of Section 6.

The corollary is true and nearly useless, and the reason is the subject of the next section. A simulation statement says that the adversary learns nothing beyond the leakage profile; it says nothing about what the profile, taken as a whole, determines.

5.6 Influence does not change what leaks

Observation 34. Theorem 28 holds for every approval path Φ, hence for every value of γ. The leakage profile of a gateway is therefore *not* improved by tightening the approval path.

This is worth stating plainly because it contradicts a common deployment intuition, namely that human-in-the-loop confirmation is a confidentiality control. It is not. What it controls is *rate and selection*:

Corollary 35 (Influence sets the rate, not the profile)**.** *Let $\mathcal{A}$ have influence $\gamma > 0$ and suppose an attack requires the execution of m adversary-chosen calls. Then the number of proposals until the attack completes is stochastically dominated by a negative binomial with parameters (m, γ), so its expectation is at most m/γ; and the profile reached is independent of γ.*

Proof. By Definition 5, at every step and conditioned on any view, the next proposal is executed with probability at least γ. Couple the experiment step by step with independent Bernoulli(γ) trials, declaring a success whenever the coupled trial succeeds; the coupling is valid precisely because the conditional probability dominates γ uniformly, and it requires no independence between steps of the experiment itself — which would be false, since $\mathsf{G}_{\mathsf{adm}}$ carries budget state. The proposal count of the coupled process is NegBin(m, γ) with mean m/γ and dominates the true count. The profile claim is Observation 34. □

Two operational readings follow. Approval paths buy *time and detectability*, not confidentiality; they should be evaluated as rate-limiters and instrumented accordingly. And an attack that needs only a handful of calls — Theorem 41 needs two — is barely slowed even by γ an order of magnitude below one.

6 Leakage Composition

Theorem 28 says the model host learns nothing beyond $(\mathcal{L}_{\mathsf{schema}}, \mathcal{L}_{\mathsf{token}}, \mathcal{L}_{\mathsf{agg}})$. It does not say what those three, taken together, determine. This section shows that the natural ways of bounding them — adding information budgets, and imposing per-channel safety predicates — both fail, and identifies what does not.

6.1 Information is sub-additive; that is not reassuring

The first instinct of a careful practitioner is to account leakage additively: estimate the bits disclosed by tokenisation, the bits disclosed by released aggregates, add, compare to a budget. The arithmetic is sound in a direction that is of no help.

Proposition 36 (Exact composition identity)**.** *Fix a query sequence q and regard D as a random variable. Let V_1 be the scoped equality pattern $\mathsf{eq}(D, q)$ and V_2 the vector of values released by q — the deterministic components of $\mathcal{L}_{\mathsf{token}}$ and $\mathcal{L}_{\mathsf{agg}}$ respectively, obtained by conditioning on the approval path's randomness and discarding timestamps. Then*

$$I(D; V_1, V_2 \mid q) \;=\; I(D; V_1 \mid q) + I(D; V_2 \mid q) - I(V_1; V_2 \mid q),$$

and in particular $I(D; V_1, V_2 \mid q) \le I(D; V_1 \mid q) + I(D; V_2 \mid q)$*, with equality if and only if* V_1 *and* V_2 *are independent given* q*.*

Proof. Determinism gives $H(V_i \mid D, q) = 0$, hence $I(D; V_i \mid q) = H(V_i \mid q)$ and $I(D; V_1, V_2 \mid q) = H(V_1, V_2 \mid q)$. Substituting $H(V_1, V_2 \mid q) = H(V_1 \mid q) + H(V_2 \mid q) - I(V_1; V_2 \mid q)$ gives the identity; non-negativity of mutual information gives the inequality, with equality exactly when $I(V_1; V_2 \mid q) = 0$. □

Remark 37. So super-additive leakage, in the information-theoretic sense, cannot occur in this architecture: two deterministic views of the same database are redundant, never synergistic in total volume. An additive budget is therefore a *valid upper bound* on total disclosed information, and looks conservative for exactly that reason.

The instance of Theorem 41 shows why the direction of the inequality is beside the point. Under the prior fixed in Construction 39, the token view carries *no* information about the sealed attribute: $I(x; V_1) = 0$. An additive account of the two channels therefore returns precisely the aggregate channel's own figure — the entropy of two released sums — and attributes nothing at all to tokenisation. A single record is nonetheless reconstructed exactly. The measure is not merely loose here; it is insensitive to the failure, because the quantity that fails is not an amount of information but the identifiability of a particular functional.

Under adaptive querying the accountant's position is worse still, since q is itself a function of the adversary's view and the conditioning above is unavailable.

The conclusion to draw is not that composition is benign but that total information is the wrong statistic. A bounded and even small number of bits can determine exactly the fact one was trying to protect.

6.2 Safety predicates do not compose

Deployments state their guarantees not as bit counts but as predicates. We fix the two that actually appear in practice.

Definition 38 (Per-channel safety predicates)**.** *Token-side:* S^k_{tok} holds if the token view is k-anonymous with respect to the declared quasi-identifier set QI, i.e. every combination of QI values occurring in D occurs in at least k records.

Aggregate-side: S^k_{agg} holds if every released aggregate has query-set size in $[k,\ n-k]$ — the classical minimum-query-set-size (MQSS) control [1, 20].

Construction 39 (Separation instance)**.** Let $n = kM$ for integers k, M with $k \le n/4$ and $n \ge 4$. Define The schema publishes nothing in the clear, so $\Sigma_{\texttt{open}} = \emptyset$. Three attributes are `token`-class, $\Sigma_{\texttt{token}} = \{\mathrm{ID}, \mathrm{ORG}, \mathrm{REGION}\}$, of which ID is a primary key and the declared quasi-identifier set is $\mathrm{QI} = \{\mathrm{ORG}, \mathrm{REGION}\}$; the internal identifier is excluded from QI because it appears in no auxiliary source. One attribute is `sealed`, $\Sigma_{\texttt{sealed}} = \{\mathrm{SAL}\}$, with value vector $x = (x_1, \ldots, x_n) \in \mathbb{Z}^n$. The n records are partitioned by their $(\mathrm{ORG}, \mathrm{REGION})$ values into M classes of exactly k records each, which is what makes the token view k-anonymous.

The interface offers two tools. A lookup tool returns, per record, the token triple $\mathsf{Tok}(\mathrm{ID})$, $\mathsf{Tok}(\mathrm{ORG})$, $\mathsf{Tok}(\mathrm{REGION})$. An aggregate tool computes $\mathrm{SUM}(\mathrm{SAL})$ over predicates $P \in \mathcal{P}$, where $\mathcal{P}$ is the set of Boolean combinations of atoms $(A = \tau)$ for $A \in \Sigma_{\texttt{token}}$. Admission is MQSS with parameter k: a query is answered exactly when its query set has size in $[k, n-k]$.

Finally we fix a prior on D under which the entries of x are i.i.d. and independent of the ID, ORG and REGION values.

Remark 40. The independence hypothesis is stated because parts (c) and (d) below are claims about a posterior and are therefore claims about a prior; without one they are not well formed. It is also the *favourable* case for the deployment: correlation between salary and organisation would only give the adversary more. We adopt it so that the separation cannot be attributed to a correlation the gateway failed to consider, and note that it puts the instance squarely in the additive regime of Proposition 36.

Every element of Construction 39 is ordinary. Tokenised primary keys are what tokenisation gateways produce; k-anonymity assessed over declared quasi-identifiers, with internal identifiers excluded because they do not appear in auxiliary sources, is how pseudonymisation risk assessments are written; MQSS is the standard aggregate control.

Theorem 41 (Safety predicates do not compose)**.** *In Construction 39, for every $k \le n/4$ and every target record i^*:*

(a) S^k_{tok} *holds;*

(b) every query the attack below submits *satisfies* S^k_{agg} *and is admitted (the singleton predicate C is used only to define them, and is never submitted);*

(c) an adversary holding only $\mathcal{L}_{\mathsf{token}}$ *has posterior on* x_{i^*} *equal to its prior;*

(d) an adversary holding only $\mathcal{L}_{\mathsf{agg}}$*, with no lookup response, has posterior on* x_{i^*} *equal to its prior, up to a negligible term, because every predicate it can realise is rejected;*

(e) an adversary holding both recovers x_{i^*} exactly*, using two aggregate queries — or four, if the predicate language is restricted as in Remark 42.*

Proof. (a) holds by construction: each $(\mathrm{ORG}, \mathrm{REGION})$ class has exactly k records.

(c) $\mathcal{L}_{\mathsf{token}}$ is a function of the ID, ORG and REGION values only (Definition 20); SAL is `sealed` and appears in no lookup response. By the independence hypothesis of Construction 39, $\mathcal{L}_{\mathsf{token}} \perp x$, so the posterior is the prior.

(d) $\mathcal{P}$ is generated by atoms $(A = \tau)$ with τ a token. By scope-confinement (Definition 1) the tokens are the image of a random injection into $\{0,1\}^\ell$, about which an adversary holding no lookup response has no information. If it submits $q = \mathrm{poly}(\kappa)$ predicates, the probability that any atom it names matches a token in use is at most $qn|\Sigma_{\mathtt{token}}|/2^\ell$, negligible because $\ell = \omega(\log \kappa)$. Conditioned on no match, every atom is false, so every predicate it can write is equivalent to $\top$ or to $\bot$. Their query sets have sizes n and 0, both outside $[k, n-k]$, so MQSS rejects both — and rejects them for a reason that does not depend on x. The adversary therefore receives no x-dependent response at all, and its posterior is its prior.

(b) and (e). The adversary first issues lookups and thereby learns, for every record, its ID token; write τ^* for the token of i^* and $C = (\mathrm{ID} = \tau^*)$, so that $|C(D)| = 1$ and $\mathrm{SUM}(C) = x_{i^*}$. Note that C itself is rejected, since $1 < k$.

Choose a set $\mathcal{T}$ of $\lfloor n/2 \rfloor$ ID tokens with $\tau^* \notin \mathcal{T}$, and put $T = (\mathrm{ID} \in \mathcal{T})$ and $T' = T \vee C$. Issue $\mathrm{SUM}(T')$ and $\mathrm{SUM}(T)$. Their query sets have sizes $\lfloor n/2 \rfloor + 1$ and $\lfloor n/2 \rfloor$, both in $[k, n-k]$ whenever $k \le n/4$ and $n \ge 4$, so both are admitted, giving (b). Since $T'(D) = T(D) \cup \{i^*\}$ disjointly,

$$x_{i^*} = \mathrm{SUM}(T') - \mathrm{SUM}(T), \tag{1}$$

which is (e). □

Remark 42 (Pairwise set-difference control does not help). Eq. (1) invites an obvious patch: reject a query whose set differs from that of a previously admitted query in fewer than k records. Two things are wrong with it. First, it is no longer MQSS — it is a condition on the *history*, which is already a

concession to the argument of this section. Second, it fails at four queries. Assume $n \geq 3k+1$, let $S \ni i^*$ with $|S| = k+1$, and let $X, Y \subseteq [n] \setminus S$ be disjoint with $|X| = |Y| = k$; all are expressible as disjunctions of ID atoms. Issue

$$A = S \cup X, \qquad B = (S \setminus \{i^*\}) \cup Y, \qquad X, \qquad Y.$$

Then $\mathrm{SUM}(A) - \mathrm{SUM}(X) = \mathrm{SUM}(S)$ and $\mathrm{SUM}(B) - \mathrm{SUM}(Y) = \mathrm{SUM}(S) - x_{i^*}$, so

$$x_{i^*} = \mathrm{SUM}(A) - \mathrm{SUM}(X) - \mathrm{SUM}(B) + \mathrm{SUM}(Y). \tag{2}$$

Query sizes are $2k+1, 2k, k, k$, all in $[k, n-k]$; and every pairwise symmetric difference — $|A \triangle B| = 2k+1$, $|A \triangle X| = k+1$, $|A \triangle Y| = 3k+1$, $|B \triangle X| = 3k$, $|B \triangle Y| = k$, $|X \triangle Y| = 2k$ — is at least k. A pairwise rule constrains differences of *two* rows; the attack uses a combination of four. Patching it to triples invites a five-query variant, and so on: only a condition on the whole row space closes the family, which is Definition 54.

Remark 43 (What exactly is disclosed). The attack yields x_{i^*} exactly and attributes it to a token identity. Re-identifying that token as a natural or legal person is a separate step, bounded by k-anonymity to a class of k candidates. This distinction matters legally and should not be elided: what the deployment documented was k-anonymity of the quasi-identifiers, and what the composite adversary obtains is *exact value disclosure* attached to a k-set — a strictly different, and in most sensitive-attribute settings strictly worse, exposure than the guarantee as written. A homogeneous class makes the two coincide [49].

Example 44. With $n = 10{,}000$ customer records and $k = 500$ — a setting most practitioners would describe as conservative — two admitted aggregate queries, each covering roughly 5,000 records, recover one customer's exact balance; four queries do so even under a pairwise set-difference control.

Remark 45 (The instance is not fragile). Construction 39 uses a tokenised primary key because that is the common case, but point separation is all the attack needs. Removing ID and adding $t = \lceil \log_2 n \rceil$ binary `token`-class attributes outside QI (internal flags, product holdings, consent bits) restores it whenever the induced bit patterns are distinct: the token view reveals each record's pattern up to per-column relabelling, which then suffices to write C. Conversely, no choice of k defeats the attack, since k enters only through the requirement $k \leq n/4$.

6.3 The token channel is the enabling condition

Part (d) of Theorem 41 is the structural point, and it generalises beyond the instance.

Theorem 46 (Vocabulary enabling)**.** *Let* G *be well-formed with* $\Sigma_{\texttt{open}} = \emptyset$ *and predicate language* $\mathcal{P}$ *generated by atoms* $(A = \tau)$, $A \in \Sigma_{\texttt{token}}$. *Let* $\mathcal{A}$ *be any* PPT *adversary with access to the aggregate interface but to no lookup response, issuing* $q = \mathrm{poly}(\kappa)$ *calls. Then* $\mathcal{A}$*'s view is computationally indistinguishable from the view of an adversary that submits only the constant predicates* $\top$ *and* $\bot$.

Proof. By Definition 1 the tokens in use are the image of a uniformly random injection π into $\{0,1\}^\ell$, and $\mathcal{A}$'s view is independent of π's image except through its own queries. Let $N \leq n \cdot |\Sigma_{\texttt{token}}|$ bound the number of tokens in use. For each atom $\mathcal{A}$ submits, the probability that the named string lies in the image is at most $N/2^\ell$; by a union bound over the at most $\mathrm{poly}(\kappa)$ atoms it submits, the probability of any match is negligible, using $\ell = \omega(\log \kappa)$. Conditioned on no match every atom is false, so every predicate is equivalent to a constant and responses depend only on $\top$ and $\bot$. Indistinguishability follows; a distinguisher with non-negligible advantage would break scope-confinement. □

Corollary 47 (Asymmetry of the two channels)**.** *Under the hypotheses of Theorem 46, the aggregate channel discloses nothing about* $\Sigma_{\texttt{sealed}}$ *beyond constant-predicate statistics unless the token channel has first supplied vocabulary. The token channel is therefore not an additional summand in a leakage budget; it is a precondition on which the aggregate channel's leakage is contingent.*

Remark 48 (The hypothesis $\Sigma_{\texttt{open}} = \emptyset$ is load-bearing). A single `open`-class attribute supplies predicate vocabulary with no token at all, and Theorem 46 then says nothing. The general and honest statement is about the *exposed vocabulary* — open literals together with observed tokens — and it is this joint object, not the token channel alone, that the aggregate channel's leakage is contingent on. We state the theorem in the pure case because that is the case tokenisation gateways are sold as achieving; a deployment with open attributes has a correspondingly weaker claim, and should compute the vocabulary its catalogue already grants before counting the tokens it discloses.

This has an immediate and uncomfortable design consequence. The two modules were separated precisely so that each could be reasoned about against its own adversary (Table 2), and that separation is sound for the marginal analyses. It is exactly what makes the joint analysis invisible: $\mathsf{G}_{\mathsf{sem}}$ has no view of aggregate budgets, $\mathsf{G}_{\mathsf{com}}$ has no view of disclosed vocabulary, and $\mathsf{G}_{\mathsf{adm}}$ — the only module that sees both — is conventionally configured to count queries.

6.4 Counting queries is the wrong budget

We now show that the control $\mathsf{G}_{\mathsf{adm}}$ conventionally applies cannot bound reconstruction, for reasons independent of the particular attack above.

Fix a `sealed` attribute with value vector $x \in \mathbb{Q}^n$. Let $P_1, \ldots, P_q$ be the query sets of admitted SUM queries and let $M \in \{0,1\}^{q \times n}$ have rows $\mathbf{1}_{P_j}$, so the released values are $s = Mx$.

Theorem 49 (Rank criterion)**.** *The view determines* x_{i^*} *— that is,* $x'_{i^*} = x_{i^*}$ *for every* x' *consistent with* s *— if and only if* $e_{i^*} \in \mathrm{rowsp}(M)$*. More generally, the set of linear functionals of* x *determined by the view is exactly* $\mathrm{rowsp}(M)$*, of dimension* $\mathrm{rank}(M)$*.*

Proof. The consistent set is the affine subspace $x + \ker M$. For $z \in \ker M$, $\langle e_{i^*}, x+z\rangle = x_{i^*} + \langle e_{i^*}, z\rangle$, so x_{i^*} is constant on the consistent set iff $e_{i^*} \perp \ker M$, i.e. iff $e_{i^*} \in (\ker M)^{\perp} = \mathrm{rowsp}(M)$. The general statement is the same computation with e_{i^*} replaced by an arbitrary functional. □

Remark 50 (The criterion is necessary, not sufficient, for safety). Theorem 49 characterises determination in the unconstrained rational relaxation. Real deployments constrain x to a bounded integer range, which can determine x_{i^*} even when $e_{i^*} \notin \mathrm{rowsp}(M)$, by integer programming or lattice reduction; and releasing noisy rather than exact sums does not restore safety once the number of queries grows, by Dinur and Nissim [21] and its strengthenings [25]. The span condition of Definition 54 is therefore necessary for safety, not sufficient. This asymmetry is the right one for a control to have: a gateway that satisfies it may still be attackable, but a gateway that violates it is attackable by linear algebra alone.

Theorem 51 (MQSS does not bound rank)**.** *Let* $n \geq 2$ *and* $1 \leq k \leq n/2$*. There exists* $M \in \{0,1\}^{n \times n}$ *whose every row has weight exactly* k *and with* $\mathrm{rank}(M) = n$*. Consequently MQSS with parameter* k *admits a sequence of* n *queries that determines every entry of* x*.*

Proof. Let $W_k \subseteq \{0,1\}^n$ be the set of all indicator vectors of k-subsets of $[n]$; each has weight k and, since $k \leq n/2 \leq n-k$, MQSS-k admits every one of them. We show $\mathrm{span}\, W_k = \mathbb{R}^n$, from which n linearly independent members can be extracted to form M.

Fix $i \neq j$. Because $1 \leq k \leq n-1$ there is a k-set S with $i \in S$ and $j \notin S$. Put $S' = (S \setminus \{i\}) \cup \{j\}$, also a k-set. Then $\mathbf{1}_S - \mathbf{1}_{S'} = e_i - e_j \in \operatorname{span} W_k$. Hence span W_k contains every $e_i - e_j$, and therefore the whole hyperplane $H = \{v : \sum_i v_i = 0\}$, of dimension $n-1$. It also contains $\mathbf{1}_S$, whose coordinate sum is $k \geq 1$, so $\mathbf{1}_S \notin H$. Therefore span $W_k = \mathbb{R}^n$ and $x = M^{-1}s$. □

Remark 52 (An explicit witness). The proof is non-constructive only in the choice of basis. When $\gcd(k,n) = 1$ one may take M to be the circulant whose first row is $\mathbf{1}_{\{0,\ldots,k-1\}}$ over $\mathbb{Z}_n$: its eigenvalues are $\lambda_\ell = \sum_{j<k} \omega^{j\ell} = (\omega^{k\ell} - 1)/(\omega^\ell - 1)$ for $\omega = e^{2\pi i/n}$, with $\lambda_0 = k \neq 0$, and for $\ell \neq 0$ vanishing only if $n \mid k\ell$, which coprimality forbids. For $\gcd(k,n) = g > 1$ the circulant has rank $n-(g-1)$ and must be supplemented, which is why we prove the general case by the elementary argument instead.

Corollary 53 (The controls are orthogonal)**.** *MQSS constrains* $\|\mathbf{1}_{P_j}\|_1$*, a property of individual rows; determination is a property of* rowsp(M)*. Theorem 51 exhibits matrices maximising the latter while satisfying the former, so no bound on row weights implies any bound on rank. A gateway that limits the number or the size of admitted aggregate queries is not measuring the quantity that governs disclosure.*

6.5 What a composition-aware admission module must track

It is tempting to read Theorem 51 as saying that the budget unit should be rank(M). It should not, and the reason is instructive: the two-query attack of Theorem 41 produces $M = [\mathbf{1}_{T'}; \mathbf{1}_T]$ with $\operatorname{rank}(M) = 2$, and $e_{i^*} = \mathbf{1}_{T'} - \mathbf{1}_T \in \operatorname{rowsp}(M)$. Any bound $\beta_{\text{rank}} \geq 2$ admits it. Rank is a *dimension*; Theorem 49 is a statement about *which vectors* lie in the row space. Bounding the first does not bound the second.

The correct control is the row-space lift of MQSS itself.

Definition 54 (Span condition)**.** Admitted query matrix M satisfies SPAN$_k$ if every non-zero $v \in \operatorname{rowsp}(M)$ has $|\operatorname{supp}(v)| \geq k$.

Proposition 55 (SPAN$_k$ is the right generalisation)**.** SPAN$_k$ *implies the lower bound of MQSS-k on every admitted query; for* $k \geq 2$ *it implies* $\operatorname{rank}(M) \leq n-k+1$*. The converse implications fail: MQSS-k implies neither* SPAN$_k$ *(Theorem 51) nor the non-determination of a single record (the two-query attack of Theorem 41).*

Proof. Each row of M lies in rowsp(M), so SPAN$_k$ bounds its weight below by k; the upper bound $|P_j| \leq n-k$ is a separate condition on rows and we retain it explicitly. If some x_i were determined then $e_i \in \operatorname{rowsp}(M)$ by Theorem 49, and $|\operatorname{supp}(e_i)| = 1 < k$ contradicts SPAN$_k$. For the rank bound, observe that SPAN$_k$ says exactly that rowsp(M), viewed as a linear code of length n, has minimum Hamming weight at least k; the Singleton bound, whose proof is valid over any field, then gives $\dim \operatorname{rowsp}(M) \leq n-k+1$. □

Remark 56. The rank bound is worth noticing for two reasons. It shows that the right condition does constrain rank after all — just not the other way round, which is the asymmetry the two-query attack exhibits. And it makes the analogy underlying Remark 58 exact rather than suggestive: enforcing SPAN$_k$ is deciding the minimum distance of the code rowsp(M).

Definition 57 (Disclosure budget)**.** A *disclosure budget* is a triple $\mathfrak{B} = (\beta_{\text{voc}}, k_{\text{span}}, \beta_{\text{rev}})$ maintained per (principal, scope, epoch): The first component, β_{voc}, bounds the exposed vocabulary. Its crude form counts distinct tokens disclosed; the meaningful form bounds the *separating power* of the predicate algebra $\mathcal{B}$ those tokens generate, which we take to be $\log_2$ of the number of cells in the partition

of $[n]$ that $\mathcal{B}$ induces. Point separation is the extreme value $\log_2 n$, and Proposition 61 says that reaching it is enough on its own to defeat the aggregate-side control. The second, k_{span}, is the parameter of Definition 54: the admission module maintains $\text{rowsp}(M)$ per `sealed` attribute and refuses any query after which $\text{SPAN}_{k_{\text{span}}}$ would fail. The third, β_{rev}, bounds explicit Reveal operations, and is the only one of the three that most deployments already have. A budget is *composition-aware* when the admission decision is a function of the joint state — the exposed vocabulary together with $\text{rowsp}(M)$ — rather than of either component alone.

Remark 58 (Enforcement is easy at $k = 2$ and hard beyond). SPAN_2 — equivalently, "no single record is determined" — is decidable in polynomial time: a non-zero vector of support 1 in a subspace is a multiple of some e_i, so it suffices to test $e_i \in \text{rowsp}(M)$ for each i, which is n rank computations and can be maintained incrementally. This is a control an admission module can actually implement, and we recommend it as the floor.

General SPAN_k is another matter. By Proposition 55 it is the minimum-distance question for the code $\text{rowsp}(M)$ over $\mathbb{Q}$. We do not give a reduction and do not assert hardness for this exact formulation; but every neighbouring problem is NP-hard. Deciding the minimum distance of a linear code over a finite field is NP-hard [60]; finding a sparsest basis of a null space is NP-hard [15]; and auditing sum queries over *Boolean* attributes — deciding whether answering the next query enables a compromise — is NP-hard [47]. The Boolean restriction in the last of these is not a technicality to be waved away: it is precisely the bounded-domain setting that Remark 50 identifies as the realistic one, and in which Theorem 49's polynomial rank test ceases to be exact. A deployment should therefore expect to enforce SPAN_2 exactly and to approximate conservatively above it, for instance by restricting $\mathcal{P}$ so that $\text{rowsp}(M)$ stays inside a family whose minimum support is computable by construction. This is a further argument for C5 of Appendix A.

Proposition 59 (The deployed conjunction is insufficient)**.** *The admission rule $S^k_{\text{tok}} \wedge S^k_{\text{agg}}$ — k-anonymity of the token view together with MQSS-k — fails to prevent the attack of Theorem 41 for every $k \le n/4$.*

Proof. Immediate from Theorem 41(a),(b): both conjuncts hold throughout the attack at parameter k while the attack succeeds. □

Remark 60. We state this for the specific pair that deployments use, not for all conjunctions of per-channel predicates. Some per-channel predicates do stop the attack — "disclose at most one token" is one — at the cost of forbidding the workload the gateway exists to serve. The content of the proposition is that the conjunction people actually deploy is not among them, and that nothing in either conjunct is sensitive to the other channel's state.

Proposition 61 (Separation threshold)**.** *Suppose $\mathcal{P}$ is closed under disjunction and the exposed vocabulary generates a predicate algebra that separates the points of D. Then for every $k \le n/4$ and every record there is an MQSS-k-admitted attack of two queries, and of four under a pairwise set-difference control. Point separation obtains as soon as the lookup interface exposes a tokenised primary key, or any set of `token`-class attributes jointly unique per record; with t binary `token`-class attributes it obtains once $t \ge \lceil \log_2 n \rceil$ and the induced patterns are distinct.*

Proof. Point separation yields, for the target, a predicate C with $|C(D)| = 1$; closure under disjunction makes the tracker set T of Theorem 41 and the sets A, B, X, Y of Remark 42 expressible. The arguments of Theorem 41(b),(e) and Remark 42 then apply verbatim. □

Remark 62. Dropping $\vee$-closure does not make a gateway safe; it makes the question a computation about $\mathcal{P}$, namely whether the algebra contains sets of the required sizes and differences. Constraint C5 of Appendix A exists so that this computation is possible at all.

Proposition 61 is the sharpest statement we can make for practice: *tokenising the primary key, the single most common gateway design decision, is precisely what collapses the aggregate-side control.* The two mechanisms were adopted for unrelated reasons, each is defensible in isolation, and together they are self-defeating.

Three levers remain. Their relative effectiveness is an empirical question, and we set out below what each one acts on.

Scope. Session- or epoch-scoped tokens bound the vocabulary that can be accumulated within a budget window. Of the three levers this is the only one that acts on the *precondition* of Theorem 46 rather than on its consequence, and it is therefore the one whose effect compounds: an adversary that cannot carry a token across a session boundary must rebuild its predicate language each time. The cost is borne by exactly the workloads that motivate global scope, namely those that need to recognise the same entity across conversations.

Span accounting. Maintaining $\mathrm{rowsp}(M)$ and enforcing SPAN_k (Definition 54) is the control Theorem 49 licenses, and it is the one the aggregate side is currently missing. Its floor, SPAN_2, is cheap enough that we would treat it as mandatory: it is n rank tests per admitted query, maintainable incrementally. Above that floor enforcement becomes approximate (Remark 58), so the practical question is not whether to approximate but how conservatively.

Noise. Perturbing released aggregates breaks the exactness on which Theorem 49 depends and moves the deployment into the regime where differential privacy [24, 26] gives quantitative guarantees. The difficulty is not the mechanism but the population: in the low-cardinality business-entity settings these gateways serve, the noise required to protect a single firm's figure is frequently large relative to the figure itself, and the resulting answers may be useless for the purpose that justified the query.

6.6 Implications for measurement

The results above determine what a quantitative study of this architecture must measure, and rule out several things it must not report.

Marginal leakage figures are uninformative. Under the independence prior of Construction 39 the token channel's marginal contribution to sealed-attribute disclosure is zero and, by Corollary 47, the aggregate channel's is constant-predicate only; where the attributes are correlated the token channel contributes whatever the correlation supplies, which is a different quantity again. Reporting either marginal alone is not conservative, it is beside the point. Experiments must vary the two jointly and state the correlation structure of the corpus they use.

The controlling variable is vocabulary acquisition rate. Attack success should be reported against the number of distinct tokens exposed per scope, not against the number of queries issued. The two diverge sharply: a workload that issues many queries over a small vocabulary is safe in a way that a workload issuing few queries over a large one is not, and a study indexed on query count cannot see the difference.

The aggregate-side axis is the row space. Reconstruction curves should be plotted against the minimum support attained in $\mathrm{rowsp}(M)$. Reporting against query count overstates safety, and reporting against $\mathrm{rank}(M)$ misses the two-query attack entirely — a mistake we note because it is the natural reading of Theorem 51 and we made it ourselves before Theorem 49 was stated carefully.

γ belongs on the time axis, not the success axis. By Corollary 35 influence rescales elapsed proposals by at most $1/\gamma$ and leaves success probability at a given executed-query count unchanged. A study that plots success against wall-clock time or against proposals will therefore report a property of the approval path rather than of the gateway.

Population cardinality is a first-order parameter. k-anonymity over business entities behaves very differently from k-anonymity over natural persons, because entity populations are small and public directories — corporate registries, disclosure filings, industry association lists — are close to complete. A study whose corpus does not reflect that will overstate safety, and will do so in precisely the deployments these gateways are sold into.

7 What This Model Does Not Protect

A threat model is only as useful as its stated boundary. The following are outside the model, and a deployment claiming the guarantees of Section 5 while exposed to any of them is misdescribing itself.

7.1 The push path

Everything above concerns the *pull* path: data leaving the estate through tool calls the gateway mediates. Nothing here constrains the *push* path — a user pasting a customer record into a chat window, uploading a spreadsheet, or forwarding a document to a model endpoint that does not traverse the gateway. This is not a subtle residual risk; in most organisations it is the dominant one, and it is a network-egress and endpoint-control problem, not a cryptographic one. Gateway deployments that do not also close the push path have changed which channel the data leaves by, not whether it leaves.

7.2 The fully privileged insider

A principal legitimately entitled to see plaintext — a data owner, an operator of G with key access, an administrator who can alter Π — is outside the model by construction. Reveal produces an audit record for such a principal, so its actions are *attributable*, which is a real property and the one on which regulatory deterrence rests; it is not a confidentiality property. We note in particular that Definition 17 says nothing about whether Π itself is well-chosen. A policy that permits everything satisfies W2 perfectly.

7.3 An actively malicious model host

Definition 9 assumes a host that is honest as a responder. A host that deviates arbitrarily can, among other things, fabricate tool results to manipulate a human approver, emit outputs engineered to induce a user to perform a Reveal, or withhold results to probe timing. Such a host is not addressed by any result here, and we are sceptical of claims that it can be addressed at this layer at all: the host's outputs are, by construction, trusted to be semantically meaningful by the humans reading them. Attestation of the inference environment [16] changes the assumption but does not remove it.

A weaker and more tractable sub-case is worth separating: a host that is honest but whose *inputs* are attacker-controlled, i.e. indirect prompt injection [18, 36]. That case is in scope and is exactly what $\gamma \to 1$ models.

7.4 Side channels beyond the declared profile

$\mathcal{L}_{\text{pattern}}$ captures the stored-token equality pattern together with storage-layer access pattern, volume, and timing at the granularity a storage operator observes. It does not capture microarchitectural side channels, power or electromagnetic analysis, or timing at instruction granularity within G. Nor does the model provide access-pattern hiding: $\mathcal{L}_{\text{pattern}}$ is disclosed, not suppressed. A deployment requiring access-pattern confidentiality needs oblivious RAM [35, 58] or volume-hiding constructions [41], at costs this architecture does not pay.

7.5 Correctness and utility of model output

Nothing here concerns whether the model's answers are right. Tokenisation degrades answer quality in ways that vary sharply by task, and a gateway that is perfectly confidential and produces useless answers has not solved the deployment problem. Utility measurement is a separate exercise and we make no claim about it.

7.6 Training-data retention by the host

Whether the model host retains prompts and returns, and whether retained material influences future model behaviour, is governed by contract and by the host's controls, not by the gateway. What the gateway changes is the *content* of anything retained: under Theorem 28, what can be retained is bounded by $\mathcal{L}_{\text{host}}$. That is a meaningful reduction and should be stated as exactly that, not as a guarantee of non-retention. The literature on extraction from trained models [8, 46, 57] is the relevant reference for what retention would imply.

7.7 Availability

Denial of service against G, and the failure-open behaviour of approval paths under load, are out of scope. We note only that a gateway whose admission module fails open under load has converted an availability incident into a disclosure incident, and that Definition 57 state must therefore be durable across restarts.

7.8 Multi-party and federated settings

We assume a single data owner. Deployments in which several owners contribute to a shared estate — consortium arrangements, data combination under pseudonymisation regimes — raise composition questions of a different kind, since each owner's leakage profile is evaluated against a different auxiliary-knowledge assumption. We regard this as the most significant open extension.

8 Discussion

8.1 Why the modular decomposition backfires

The three-module decomposition of Section 3.4 is good engineering. Each module has one adversary, one mechanism, and one testable property, and the simulation theorems of Section 5 are provable precisely because responsibilities are separated. The same separation is what Theorem 41 exploits:

the token module decides scope without knowing what aggregates will be asked, the compute module evaluates aggregates without knowing what vocabulary has been disclosed, and the admission module — the only one positioned to see both — is conventionally configured to count queries rather than to track the joint state of Definition 57.

The general lesson is not specific to this architecture. When a system's safety argument is a conjunction of per-component arguments, the components must either be independent, which for deterministic views of a common database they never are, or the conjunction must be supplemented by an argument about the joint state. The gateway pattern makes the omission especially easy because the two components leak about *different attribute classes*: the token channel about identifiers, the aggregate channel about values. It is natural to conclude that they cannot interact. They interact through the predicate language.

8.2 What the model asks a deployment to write down

The model is usable only if a deployment can state, for its own schema: the protection class of every attribute; the declared quasi-identifier set and the auxiliary sources that justify it; the scope of every token class; the predicate language of every aggregate tool; the policy governing Reveal; and the budget of Definition 57. Our experience is that the third and fourth of these are usually undocumented, and that writing them down changes designs before any analysis is performed — typically by revealing that the predicate language is "whatever the query builder happens to emit", which by Theorem 49 is equivalent to unbounded rank.

Appendix A turns these obligations into fourteen constraints on the tool schema itself, which is the form in which they can be checked mechanically.

8.3 Relationship to pseudonymisation regimes

Tokenisation is routinely presented to regulators as pseudonymisation, and k-anonymity over declared quasi-identifiers as the accompanying risk assessment. Theorem 41 and Remark 43 show that this pairing, combined with an aggregate interface, supports exact value disclosure attached to a k-set while both stated properties continue to hold. We take no position on how any particular regulator would assess that. We do observe that the relevant statutory obligations are framed around the risk of re-identification and the adequacy of technical measures [29, 55], which are questions about the joint state, and that a risk assessment evaluating the two channels separately is not responsive to them. Appendix B gives the descriptive mapping.

9 Conclusion

We have given a threat model for privacy gateways in LLM agent systems that separates three adversaries rather than one, and a formalisation of the disclosure boundary as an explicit, gated, audited operation with a four-component leakage profile. The simulation statements (Theorems 28 and 29) are unsurprising, and deliberately so: they are the price of admission for saying anything precise afterwards.

The results that matter are negative. Total leakage is sub-additive (Proposition 36), so bit-counting budgets are conservative in the statistic they measure and silent about the one that matters. Per-channel safety predicates do not compose (Theorem 41): k-anonymity over declared quasi-identifiers and minimum-query-set-size control can both hold while two admitted queries recover a record exactly. The token channel is not an additional leakage source but the enabling condition

for the aggregate channel (Theorem 46), which means the two cannot be budgeted independently even in principle. And minimum-query-set-size control cannot bound reconstruction at all, because it constrains row weights while determination is a property of the row space (Theorems 49 and 51); the replacement is the span condition of Definition 54, exactly enforceable for single records and NP-hard-adjacent beyond.

We have also argued that the approval path, widely treated as a confidentiality control, is not one: influence γ leaves the leakage profile unchanged and rescales attack time by at most $1/\gamma$ (Corollary 35). Against a two-query attack, that is not much rescaling.

None of this says the architecture is unsound. It says which of its controls carry weight and which do not, and it identifies scope selection and span accounting as the two levers that act on the mechanism rather than on its symptoms. We also correct the budget unit that Theorem 51 appears to suggest: it is not the rank of the released query matrix, since the two-query attack has rank two, but the minimum support attained in its row space. Whether those levers can be set to values that are simultaneously safe and useful is an empirical question, and specifically an empirical question about the low-cardinality entity populations typical of enterprise deployments, where public auxiliary sources are close to complete. Section 6.6 states what such a study must measure. We have been careful to claim nothing about the magnitudes in advance.

Availability

The adversary table (Table 1), the disclosure-boundary mapping (Table 3), and the schema constraints of Appendix A are intended for direct reuse in deployment documentation and interface specifications, and may be reproduced for that purpose.

Acknowledgements

This work is a specification-stage artefact: the constraints of Appendix A were derived in order to fix the requirements clause of a tool-schema specification, and several of them exist because an earlier draft of that specification violated them.

References

[1] Nabil R. Adam and John C. Worthmann. Security-control methods for statistical databases: A comparative study. *ACM Computing Surveys*, 21(4):515–556, 1989.

[2] Anthropic. Model context protocol specification. `https://modelcontextprotocol.io`, 2024.

[3] Panagiotis Antonopoulos, Arvind Arasu, Ken Eguro, Nitish Gupta, Rajat Jain, Raghav Kaushik, Hanuma Kodavalla, Donald Kossmann, et al. Azure SQL database Always Encrypted. In *Proc. ACM SIGMOD International Conference on Management of Data*, pages 1511–1525. ACM, 2020.

[4] Mihir Bellare, Alexandra Boldyreva, and Adam O'Neill. Deterministic and efficiently searchable encryption. In *Advances in Cryptology — CRYPTO 2007*, volume 4622 of *LNCS*, pages 535–552. Springer, 2007.

[5] Vincent Bindschaedler, Paul Grubbs, David Cash, Thomas Ristenpart, and Vitaly Shmatikov. The tao of inference in privacy-protected databases. *Proceedings of the VLDB Endowment*, 11 (11):1715–1728, 2018.

[6] Laura Blackstone, Seny Kamara, and Tarik Moataz. Revisiting leakage abuse attacks. In *Proc. 27th Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2020.

[7] Dan Boneh, Eu-Jin Goh, and Kobbi Nissim. Evaluating 2-DNF formulas on ciphertexts. In *Theory of Cryptography (TCC)*, volume 3378 of *LNCS*, pages 325–341. Springer, 2005.

[8] Nicholas Carlini, Florian Tramer, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Úlfar Erlingsson, Alina Oprea, and Colin Raffel. Extracting training data from large language models. In *Proc. 30th USENIX Security Symposium*, pages 2633–2650. USENIX Association, 2021.

[9] David Cash, Paul Grubbs, Jason Perry, and Thomas Ristenpart. Leakage-abuse attacks against searchable encryption. In *Proc. 22nd ACM Conference on Computer and Communications Security (CCS)*, pages 668–679. ACM, 2015.

[10] Melissa Chase and Seny Kamara. Structured encryption and controlled disclosure. In *Advances in Cryptology — ASIACRYPT 2010*, volume 6477 of *LNCS*, pages 577–594. Springer, 2010.

[11] Tianyu Chen, Hangbo Bao, Shaohan Huang, Li Dong, Binxing Jiao, Daxin Jiang, Haoyi Zhou, Jianxin Li, and Furu Wei. THE-X: Privacy-preserving transformer inference with homomorphic encryption. In *Findings of the Association for Computational Linguistics (ACL)*, pages 3510–3520, 2022.

[12] Yu Chen, Tingxin Li, Huiming Liu, and Yang Yu. Hide and seek (HaS): A lightweight framework for prompt privacy protection, 2023. arXiv:2309.03057.

[13] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *Advances in Cryptology — ASIACRYPT 2017*, volume 10624 of *LNCS*, pages 409–437. Springer, 2017.

[14] Francis Y. Chin and Gultekin Özsoyoğlu. Auditing and inference control in statistical databases. *IEEE Transactions on Software Engineering*, SE-8(6):574–582, 1982.

[15] Thomas F. Coleman and Alex Pothen. The null space problem I: Complexity. *SIAM Journal on Algebraic and Discrete Methods*, 7(4):527–537, 1986.

[16] Victor Costan and Srinivas Devadas. Intel SGX explained, 2016. URL https://eprint.iacr.org/2016/086. Cryptology ePrint Archive, Report 2016/086.

[17] Reza Curtmola, Juan Garay, Seny Kamara, and Rafail Ostrovsky. Searchable symmetric encryption: Improved definitions and efficient constructions. In *Proc. 13th ACM Conference on Computer and Communications Security (CCS)*, pages 79–88. ACM, 2006.

[18] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024.

[19] Dorothy E. Denning and Jan Schlörer. Inference controls for statistical databases. *IEEE Computer*, 16(7):69–82, 1983.

[20] Dorothy E. Denning, Peter J. Denning, and Mayer D. Schwartz. The tracker: A threat to statistical database security. *ACM Transactions on Database Systems*, 4(1):76–96, 1979.

[21] Irit Dinur and Kobbi Nissim. Revealing information while preserving privacy. In *Proc. 22nd ACM Symposium on Principles of Database Systems (PODS)*, pages 202–210. ACM, 2003.

[22] David Dobkin, Anita K. Jones, and Richard J. Lipton. Secure databases: Protection against user influence. *ACM Transactions on Database Systems*, 4(1):97–106, 1979.

[23] Cynthia Dwork. Differential privacy. In *Automata, Languages and Programming (ICALP), Part II*, volume 4052 of *LNCS*, pages 1–12. Springer, 2006.

[24] Cynthia Dwork and Aaron Roth. The algorithmic foundations of differential privacy. *Foundations and Trends in Theoretical Computer Science*, 9(3–4):211–407, 2014.

[25] Cynthia Dwork and Sergey Yekhanin. New efficient attacks on statistical disclosure control mechanisms. In *Advances in Cryptology — CRYPTO 2008*, volume 5157 of *LNCS*, pages 469–480. Springer, 2008.

[26] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. Calibrating noise to sensitivity in private data analysis. In *Theory of Cryptography (TCC)*, volume 3876 of *LNCS*, pages 265–284. Springer, 2006.

[27] Kennedy Edemacu and Xintao Wu. Privacy preserving prompt engineering: A survey, 2024. arXiv:2404.06001.

[28] Taher El Gamal. A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE Transactions on Information Theory*, 31(4):469–472, 1985.

[29] European Parliament and Council. Regulation (EU) 2016/679 (general data protection regulation). Official Journal of the European Union L 119, 2016.

[30] European Parliament and Council. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence. Official Journal of the European Union, 2024.

[31] Financial Services Commission, Republic of Korea. Regulation on supervision of electronic financial transactions. FSC Public Notice, as amended, 2024.

[32] Benjamin Fuller, Mayank Varia, Arkady Yerukhimovich, Emily Shen, Ariel Hamlin, Vijay Gadepally, Richard Shay, John Darby Mitchell, and Robert K. Cunningham. SoK: Cryptographically protected database search. In *Proc. 38th IEEE Symposium on Security and Privacy (S&P)*, pages 172–191. IEEE, 2017.

[33] Srivatsava Ranjit Ganta, Shiva Prasad Kasiviswanathan, and Adam Smith. Composition attacks and auxiliary information in data privacy. In *Proc. 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 265–273. ACM, 2008.

[34] Craig Gentry. Fully homomorphic encryption using ideal lattices. In *Proc. 41st ACM Symposium on Theory of Computing (STOC)*, pages 169–178. ACM, 2009.

[35] Oded Goldreich. Towards a theory of software protection and simulation by oblivious RAMs. In *Proc. 19th ACM Symposium on Theory of Computing (STOC)*, pages 182–194. ACM, 1987.

[36] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90. ACM, 2023.

[37] Paul Grubbs, Kevin Sekniqi, Vincent Bindschaedler, Muhammad Naveed, and Thomas Ristenpart. Leakage-abuse attacks against order-revealing encryption. In *Proc. 38th IEEE Symposium on Security and Privacy (S&P)*, pages 655–672. IEEE, 2017.

[38] Paul Grubbs, Marie-Sarah Lacharité, Brice Minaud, and Kenneth G. Paterson. Pump up the volume: Practical database reconstruction from volume leakage on range queries. In *Proc. 25th ACM Conference on Computer and Communications Security (CCS)*, pages 315–331. ACM, 2018.

[39] Meng Hao, Hongwei Li, Hanxiao Chen, Pengzhi Xing, Guowen Xu, and Tianwei Zhang. Iron: Private inference on transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[40] Mohammad Saiful Islam, Mehmet Kuzu, and Murat Kantarcioglu. Access pattern disclosure on searchable encryption: Ramification, attack and mitigation. In *Proc. 19th Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2012.

[41] Seny Kamara and Tarik Moataz. Computationally volume-hiding structured encryption. In *Advances in Cryptology — EUROCRYPT 2019*, volume 11477 of *LNCS*, pages 183–213. Springer, 2019.

[42] Seny Kamara, Charalampos Papamanthou, and Tom Roeder. Dynamic searchable symmetric encryption. In *Proc. 19th ACM Conference on Computer and Communications Security (CCS)*, pages 965–976. ACM, 2012.

[43] Georgios Kellaris, George Kollios, Kobbi Nissim, and Adam O'Neill. Generic attacks on secure outsourced databases. In *Proc. 23rd ACM Conference on Computer and Communications Security (CCS)*, pages 1329–1340. ACM, 2016.

[44] Krishnaram Kenthapadi, Nina Mishra, and Kobbi Nissim. Simulatable auditing. In *Proc. 24th ACM Symposium on Principles of Database Systems (PODS)*, pages 118–127. ACM, 2005.

[45] Daniel Kifer and Ashwin Machanavajjhala. No free lunch in data privacy. In *Proc. ACM SIGMOD International Conference on Management of Data*, pages 193–204. ACM, 2011.

[46] Siwon Kim, Sangdoo Yun, Hwaran Lee, Martin Gubri, Sungroh Yoon, and Seong Joon Oh. ProPILE: Probing privacy leakage in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[47] Jon Kleinberg, Christos Papadimitriou, and Prabhakar Raghavan. Auditing boolean attributes. In *Proc. 19th ACM Symposium on Principles of Database Systems (PODS)*, pages 86–91. ACM, 2000.

[48] Brian Knott, Shobha Venkataraman, Awni Hannun, Shubho Sengupta, Mark Ibrahim, and Laurens van der Maaten. CrypTen: Secure multi-party computation meets machine learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[49] Ashwin Machanavajjhala, Johannes Gehrke, Daniel Kifer, and Muthuramakrishnan Venkitasubramaniam. ℓ-diversity: Privacy beyond k-anonymity. In *Proc. 22nd IEEE International Conference on Data Engineering (ICDE)*, page 24. IEEE, 2006.

[50] National Institute of Standards and Technology. Artificial intelligence risk management framework (AI RMF 1.0). NIST AI 100-1, 2023.

[51] Muhammad Naveed, Seny Kamara, and Charles V. Wright. Inference attacks on property-preserving encrypted databases. In *Proc. 22nd ACM Conference on Computer and Communications Security (CCS)*, pages 644–655. ACM, 2015.

[52] OWASP Foundation. OWASP top 10 for large language model applications. https://owasp.org/www-project-top-10-for-large-language-model-applications/, 2025.

[53] Personal Information Protection Commission, Republic of Korea. Guidelines on pseudonymous information processing. PIPC Guideline, 2024.

[54] Raluca Ada Popa, Catherine M. S. Redfield, Nickolai Zeldovich, and Hari Balakrishnan. CryptDB: Protecting confidentiality with encrypted query processing. In *Proc. 23rd ACM Symposium on Operating Systems Principles (SOSP)*, pages 85–100. ACM, 2011.

[55] Republic of Korea. Personal information protection act. Act No. 19234, as amended, 2023.

[56] Republic of Korea. Use and protection of credit information act. Consolidated as amended, 2024.

[57] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. Membership inference attacks against machine learning models. In *Proc. 38th IEEE Symposium on Security and Privacy (S&P)*, pages 3–18. IEEE, 2017.

[58] Emil Stefanov, Marten van Dijk, Elaine Shi, T.-H. Hubert Chan, Christopher Fletcher, Ling Ren, Xiangyao Yu, and Srinivas Devadas. Path ORAM: An extremely simple oblivious RAM protocol. *Journal of the ACM*, 65(4):18:1–18:26, 2018.

[59] Latanya Sweeney. k-anonymity: A model for protecting privacy. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 10(5):557–570, 2002.

[60] Alexander Vardy. The intractability of computing the minimum distance of a code. *IEEE Transactions on Information Theory*, 43(6):1757–1766, 1997.

[61] Yifan Yao, Jinhao Duan, Kaidi Xu, Yuanfang Cai, Zhibo Sun, and Yue Zhang. A survey on large language model (LLM) security and privacy: The good, the bad, and the ugly. *High-Confidence Computing*, 4(2):100211, 2024.

[62] Yupeng Zhang, Jonathan Katz, and Charalampos Papamanthou. All your queries are belong to us: The power of file-injection attacks on searchable encryption. In *Proc. 25th USENIX Security Symposium*, pages 707–720. USENIX Association, 2016.

A Tool-Schema Constraints

The theorems of Section 5 hold for well-formed gateways (Definition 17). Well-formedness is a property of the *interface*, so it can be discharged by constraining the tool schema rather than by auditing the implementation. Table 4 states fourteen such constraints. They are written to be mechanically checkable against a schema document and are intended to form the requirements clause of an interface specification.

Each constraint names the leakage component it governs and the result it is tied to. The "Level" column distinguishes two kinds of obligation. A constraint marked HYP is a hypothesis of a stated theorem: a gateway violating it is outside the scope of our security statements altogether. A constraint marked CTRL is a control the results *prescribe* rather than assume; violating one leaves the theorems intact and the deployment exposed to the attacks of Section 6. C7 is marked CTRL for the additional reason that, by Remark 58, it cannot in general be enforced exactly.

Three remarks on using the list.

C5 and C7 are the ones that most often force a redesign. A gateway whose aggregate tool accepts a filter expression assembled by an upstream query builder has, in effect, declared $\mathcal{P}$ to be the full Boolean algebra over all token atoms, which by Proposition 61 separates points as soon as any per-record-unique attribute is tokenised, and by Theorem 49 admits unbounded rank. Restricting $\mathcal{P}$ is usually cheaper than instrumenting span accounting, and the two are complementary rather than alternative — indeed by Remark 58 a restricted $\mathcal{P}$ may be the only way to make SPAN_k checkable at all for $k > 2$.

C8 is the constraint that is almost never present in existing designs. Lookups are treated as cheap because, considered alone, they disclose no sealed value — which is exactly Theorem 41(c). Their cost is incurred on the aggregate side.

C12 is a control rather than a hypothesis because violating it breaks no theorem; it simply hands the adversary the linkage the token class was introduced to withhold. In our experience it is violated through derived fields — an `open`-class "display label" computed from a `token`-class key — rather than deliberately.

B Regulatory Mapping

Table 5 maps each mechanism to the obligations it is commonly offered as satisfying, and states what the results of this paper do and do not support. It is descriptive: it records how these mechanisms are presented in practice and where the formal results are silent. It is not legal advice, and it does not assess any particular deployment.

Two observations about the mapping as a whole.

The Korean rows follow the structure of the Personal Information Protection Commission's guidance on pseudonymous processing [53], which is where the operative reading of the statutory terms is set out; readers applying the table outside Korea may find the risk-function vocabulary of the NIST AI Risk Management Framework [50] a closer match to their own process.

First, the obligations that the architecture discharges cleanly are the *procedural* ones: separate custody of additional information, record retention, accountability. These correspond to W1, W2, and C11, and they are satisfied by construction in a well-formed gateway. This is not a small result — procedural obligations are what most enforcement actions turn on — but it should not be reported as confidentiality.

Second, the obligations framed around the *risk of re-identification* are precisely the ones our

composition results address, and they address them negatively. A risk assessment that evaluates tokenisation and aggregate release separately, and concludes from S^k_{tok} and S^k_{agg} that residual risk is low, is making an inference that Theorem 41 invalidates. The remedy is not a different conclusion but a different object of assessment: the joint state of Definition 57.

C Notation

Table 4: Constraints on a conforming gateway tool schema.

ID	Level	Constraint	Governs	Tied to
C1	HYP	Every attribute in the published catalogue carries an explicit protection class; the default for an unannotated attribute is `sealed`.	$\mathcal{L}_{\text{schema}}$	W1
C2	HYP	Every tool argument and return value is typed by protection class. References to `token-` or `sealed`-class data occur only as tokens or ciphertext handles, never as plaintext literals.	$\mathcal{L}_{\text{token}}$	W1, W3
C3	HYP	No tool argument or return of type "free text" may carry an enclosed value without passing egress screening by G_{sem}; screening failure is a rejection, not a truncation.	$\mathcal{L}_{\text{token}}$	W3
C4	HYP	Every `token`-class attribute declares a scope function (global | tenant | session | epoch) and a rotation policy. Scope is part of the published schema, not of the implementation.	$\mathcal{L}_{\text{token}}$	W4, Theorem 28
C5	CTRL	Every aggregate tool declares a predicate language $\mathcal{P}$ as a finite, named algebra over token atoms. An aggregate tool accepting an unconstrained query string (raw SQL, arbitrary filter expressions) does not conform.	$\mathcal{L}_{\text{agg}}$	Theorem 49
C6	CTRL	Every aggregate request carries a canonical normal form of its predicate, from which G_{adm} can compute the query-set indicator vector without re-parsing.	$\mathcal{L}_{\text{agg}}$	Definition 57
C7	CTRL	G_{adm} maintains, per (principal, scope, epoch) and per `sealed` attribute, the row space of the admitted query matrix, and admits a query only if SPAN_k would still hold afterwards. At minimum it enforces SPAN_2, testing $e_i \in \text{rowsp}(M)$ for every i; query counts and rank bounds do not substitute.	$\mathcal{L}_{\text{agg}}$	Theorem 49, Definition 54
C8	CTRL	G_{adm} maintains the exposed token vocabulary per (principal, scope, epoch) and treats its growth as budgeted consumption, not as a free side effect of lookups.	$\mathcal{L}_{\text{token}}$	Theorem 46
C9	CTRL	Admission decisions are computable from the exposed state and declared policy alone. Where a data-dependent denial is unavoidable — as it is under MQSS — the schema declares it, and the denial is reported in the audit trail as a disclosure, not as a non-event.	$\mathcal{L}_{\text{agg}}$	W5
C10	HYP	Boundary crossing is a distinct tool. No tool may return a plaintext `token-` or `sealed`-class value as a side effect of another operation. Every Reveal takes an explicit purpose argument.	Δ	W2, Definition 15
C11	HYP	Every Reveal and every admitted aggregate emits an audit record binding principal, object, purpose, policy version, decision, and time. Schema, policy, and token-scope versions appear in every record so that Δ is replayable.	Δ	W2
C12	CTRL	No identifier is exposed in two protection classes. If an attribute is `token`-class, no other tool may return the same join key as `open`-class, in any projection or derived field.	$\mathcal{L}_{\text{token}}$	Proposition 61
C13	CTRL	Budget state is keyed by (principal, scope, epoch) and is durable across restarts; repeated identical requests do not reset it, and cached responses consume budget as originals do.	$\mathcal{L}_{\text{agg}}$	Definition 57
C14	HYP	The error taxonomy does not distinguish "empty result" from "policy denial" unless the distinction is declared and accounted; response latency is padded to a declared granularity for tools whose timing would otherwise separate the two.	$\mathcal{L}_{\text{agg}}$, $\mathcal{L}_{\text{pattern}}$	W6, Theorem 28

Table 5: Descriptive mapping of gateway mechanisms to regulatory obligations. "Supports" states what the formal results establish; "Does not establish" states the gap. Article numbering reflects the consolidated texts consulted and should be verified against the version in force.

Mechanism	Obligation commonly invoked	Supports	Does not establish
Tokenisation of $\Sigma_{\texttt{token}}$	PIPA 제28조의2, 제28조의4 (pseudonymised processing and safeguards); Credit Information Act 제40조의2; GDPR Art. 4(5), 25, 32 [29, 55, 56]	Equality-pattern confinement within a declared scope (Theorem 28); additional information held in $\mathcal{E}$ under W1, matching the separate-storage requirement	That re-identification is not "reasonably likely" in the sense of GDPR Recital 26: Theorem 41 shows exact value disclosure while k-anonymity holds
Token scope selection	PIPA 제28조의4 (technical safeguards); GDPR Art. 25 (by design and by default)	The only lever acting on the precondition of Theorem 46; session or epoch scope bounds accumulable vocabulary	Nothing, if scope is global: cross-session and cross-tenant linkage is then permitted by design, not by failure
Key separation and custody	PIPA 제28조의4 (1), 제29조; EFTSR 제13조 (protection of electronic data) [31]	Keys never enter $\mathcal{X}$ (W1), which is the hypothesis of Theorem 29	Protection against a principal with key custody (Section 7)
Homomorphic storage and aggregation	GDPR Art. 32(1)(a) (encryption); EFTSR 제14조, 제15조 (information-processing system protection)	Store holds ciphertext only; $\mathcal{A}_{\text{infra}}$'s view simulatable from $(\mathcal{L}_{\text{schema}}, \mathcal{L}_{\text{pattern}})$ (Theorem 29)	Confidentiality of access pattern, volume, or timing; the attack literature against that profile is mature [38, 40, 43]
Admission control (MQSS, query quotas)	PIPA 제28조의5 (prohibition on processing for the purpose of identifying a specific individual); Credit Information Act 제40조의2	Rate limiting and attributability; with span accounting (C7), a bound licensed by Theorem 49	Any bound on reconstruction, if implemented as query counting or minimum set size (Theorem 51)
Reveal gate and audit trace	PIPA 제28조의4 (2) (record retention), 제29조; GDPR Art. 5(2), 30 (accountability, records of processing); EFTSR access control and logging provisions	Δ is a complete, replayable record of clear-text crossings (W2, C11), which is the artefact these obligations require	Confidentiality; an audited disclosure is still a disclosure
Re-identification detection duty	PIPA 제28조의5 (2) (cease processing and destroy on identification)	Detection is possible only against a recorded trace; C8 and C11 supply the state such detection needs	Detection of reconstruction performed by an external adversary from admitted queries, unless rank is tracked
Risk and impact assessment	PIPA 제33조 (privacy impact assessment); GDPR Art. 35 (DPIA); EU AI Act Art. 10 (data governance for high-risk systems) [30]	A structure for the assessment: adversary table (Table 1), boundary mapping (Table 3)	An assessment conducted per channel: by Proposition 59 the conjunction of per-channel predicates is not responsive to the obligation
Egress and endpoint control	PIPA 제29조; EFTSR 제15조 (prevention of hacking and similar incidents)	Nothing in this model	The push path is out of scope (Section 7) and is generally the dominant channel

Table 6: Notation.

Symbol	Meaning
Σ, D, n	schema, database, number of records
$\mathsf{cls}(A)$	protection class of attribute A: `open`, `token`, `sealed`
QI	declared quasi-identifier set
T, Tok, $\mathcal{S}$	scoped tokeniser, its algorithm, scope set
E, $\boxplus$	additively homomorphic scheme and its operation
G; $\mathsf{G}_{\mathsf{sem}}$, $\mathsf{G}_{\mathsf{com}}$, $\mathsf{G}_{\mathsf{adm}}$	gateway; semantic, compute, admission modules
$\mathcal{T}$, $\mathcal{P}$	tool set, declared predicate language
Φ, Π	external approval predicate, gateway policy
γ	influence (Definition 5)
$\mathcal{E}$, $\mathcal{X}$, ∂	enclosed region, exposed region, boundary
Reveal, Δ	boundary-crossing operation, disclosure trace
$\mathcal{L}_{\mathsf{schema}}$, $\mathcal{L}_{\mathsf{token}}$, $\mathcal{L}_{\mathsf{agg}}$, $\mathcal{L}_{\mathsf{pattern}}$	leakage components (Definitions 19 to 24)
$\mathcal{A}_{\mathsf{host}}$, $\mathcal{A}_{\mathsf{infra}}$, $\mathcal{A}_{\mathsf{user}}$, $\mathcal{A}^{+}_{\mathsf{host}}$	adversaries (Definitions 9 to 12)
S^{k}_{tok}, S^{k}_{agg}	per-channel safety predicates
M, s, x	query-set matrix, released sums, sealed value vector
$\mathrm{rowsp}(M)$, SPAN_{k}	row space of the admitted query matrix; span condition (Definition 54)
$\mathfrak{B}$	disclosure budget (Definition 57)